Gestión de Datos: Bases de Datos Relacionales

Edgar Sarmiento Calisaya

esarmientoca@unsa.edu.pe

Universidad Nacional de San Agustín – UNSA

Faculta de Ingeniería de Producción y Servicios – FIPS

Escuela de Posgrado

Maestría en Ciencia de Datos

Breve Biografía

- **Formación:**
  - Graduación en Ingeniería de Sistemas, UNSA – Arequipa
  - Maestría en Ciencia de la Computación, UFRJ – Brasil
  - Doctorado en Ciencia de la Computación, PUC-Rio - Brasil
- **Áreas:**
  - Ingeniería de Sofware
  - Bases de Datos
- **Profesional:**
  - Ingeniero de software
  - Arquitecto de Software
  - Consultor de Software

- **Academico:**
  - **Docente Ordinario TC, UNSA - Arequipa, 2018 - …**
  - Docente Invitado, UNMSM – Lima, 2024 –
  - Docente Invitado, UCSP – Arequipa, 2026 -

Esta presentación adapta texto y figuras de documentos listados en la sección de referencias: bibliografía, recursos web, y diapositivas de diversos cursos de “Ciencia y Bases de Datos”

Agradecimiento a los profesores que disponibilizaron su material de aula.

Agenda

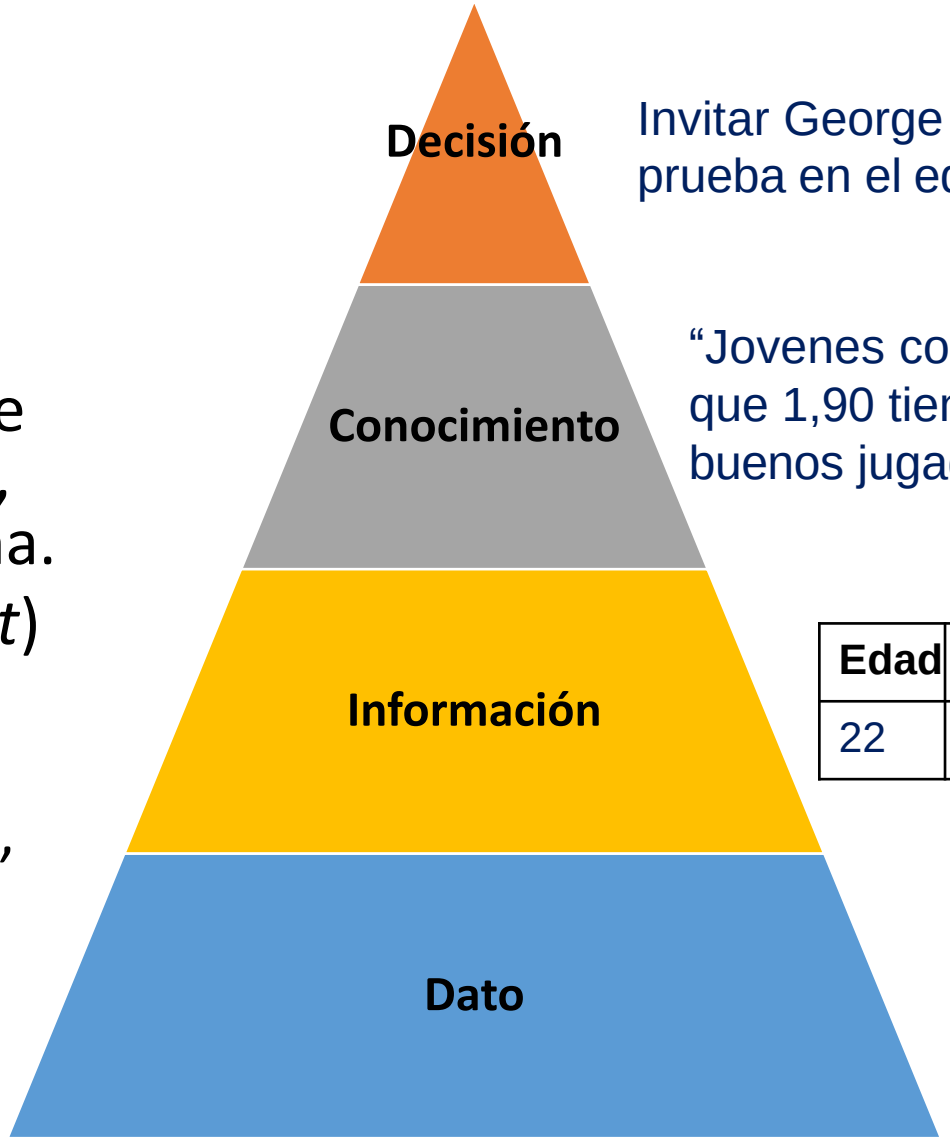
1. Introducción
    1. Motivación
    2. Conceptos
    3. ¿Por qué SGBD?
    4. Historia
2. Base de Datos
3. Modelado de Bases de Datos
    1. Conceptos: Modelo, Esquema, Instancia, Abstracción, Capas
    2. Modelo Conceptual
    3. Modelo Lógico
    4. Modelado de Bases de Datos
4. Modelo Relacional (SQL)
5. Modelado Relacional
    1. Requisitos
    2. Modelado Conceptual
    3. Modelado Lógico
    4. Modelado Físico
6. SQL
7. Conclusión

Introducción

- **Motivación**
  - La **cantidad** de información disponible crece **exponencialmente**.
    - Operaciones bancarias
    - Matricula en la universidad
    - Reserva de hotel
    - Comercio electrónico
    - Redes Sociales
    - IoT, …
  - Los **datos** y la **información** desempeñan un papel importante para las **organizaciones**, por ejemplo, al facilitar la **toma de decisiones**.
  - Es fundamental utilizar **herramientas** que simplifiquen la **gestión de datos** y la **extracción de información útil**.

Introducción

- **Conceptos**
  - **Dato**: Valor o texto aislado y **sin significado útil** o **contexto** que se pueden **almacenar**, por ejm., nombres, o direcciones.
  - **Información:** Datos organizados de forma **útil** y **significativa**, por ejm., edad y dirección de de una persona.
  - **Conocimiento: percepción** (*insight*) derivada de la información - **sabiduría**.
    - Capacidad de resolver problemas, innovar y aprender basándose en experiencias previas.
  - **Decisión: Acción** a ser tomada



Invitar George para una prueba en el equipo de básquet

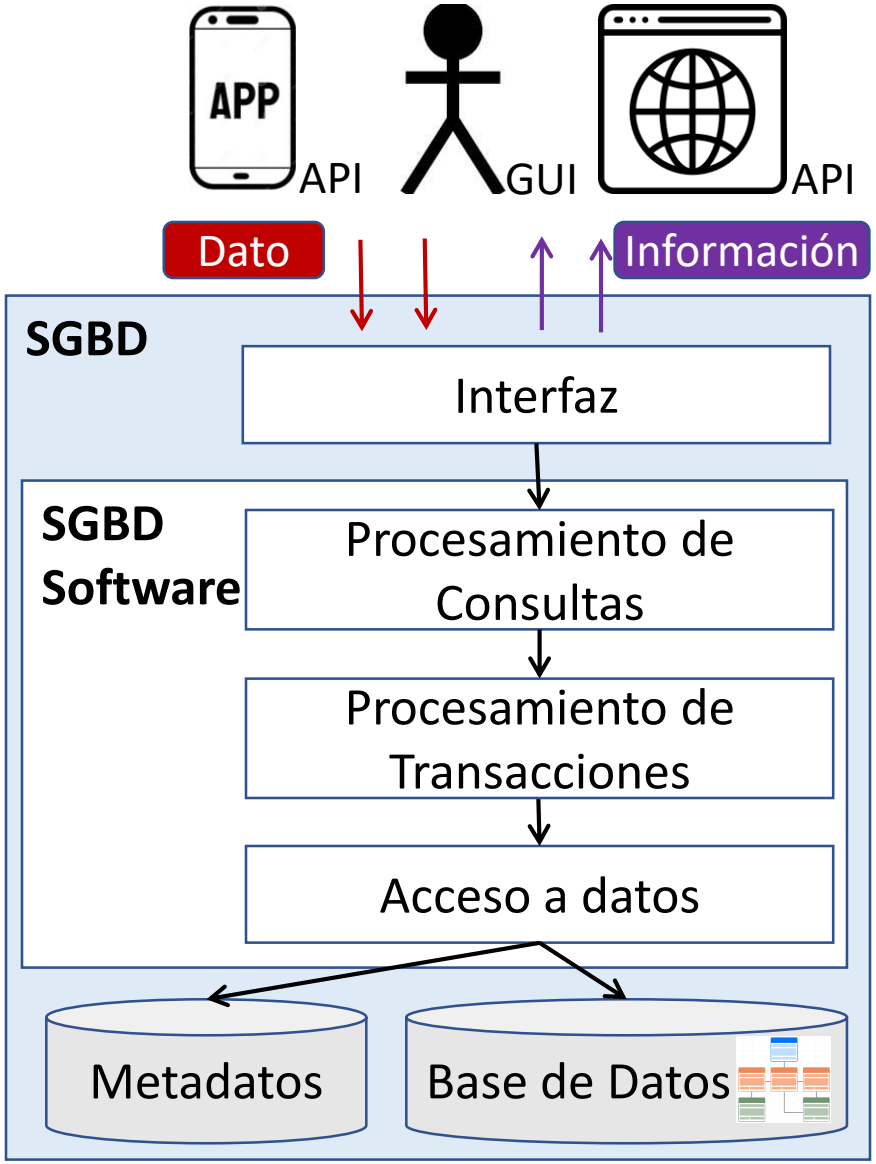
"Jovenes con altura mayor que 1,90 tienden a ser buenos jugadores de básquet"

Edad	Nombre	Altura
22	George	1,92

- George
- 22
- 1,92

Introducción

- **Conceptos**
  - **Base de datos (BD)**: conjunto de **datos lógicamente interrelacionados**, por ejemplo, una guía telefónica.
    - **metadatos** almacenados junto con los **datos**
      - **Datos** + **esquema** + **restricciones** +…
  - **Sistema de gestión de bases de datos (SGBD)**: conjunto de programas que permite la gestión de bases de datos.
    - Proporciona **seguridad**, **fiabilidad**, **concurrencia** e **integridad** de los **datos** almacenados,…
    - Permite **consultas** a los datos
  - **Sistema de Base de Datos:** Sistema de información (App, WebApp, …)
    - BDs, SGBD, hardware, personas, procesos,…

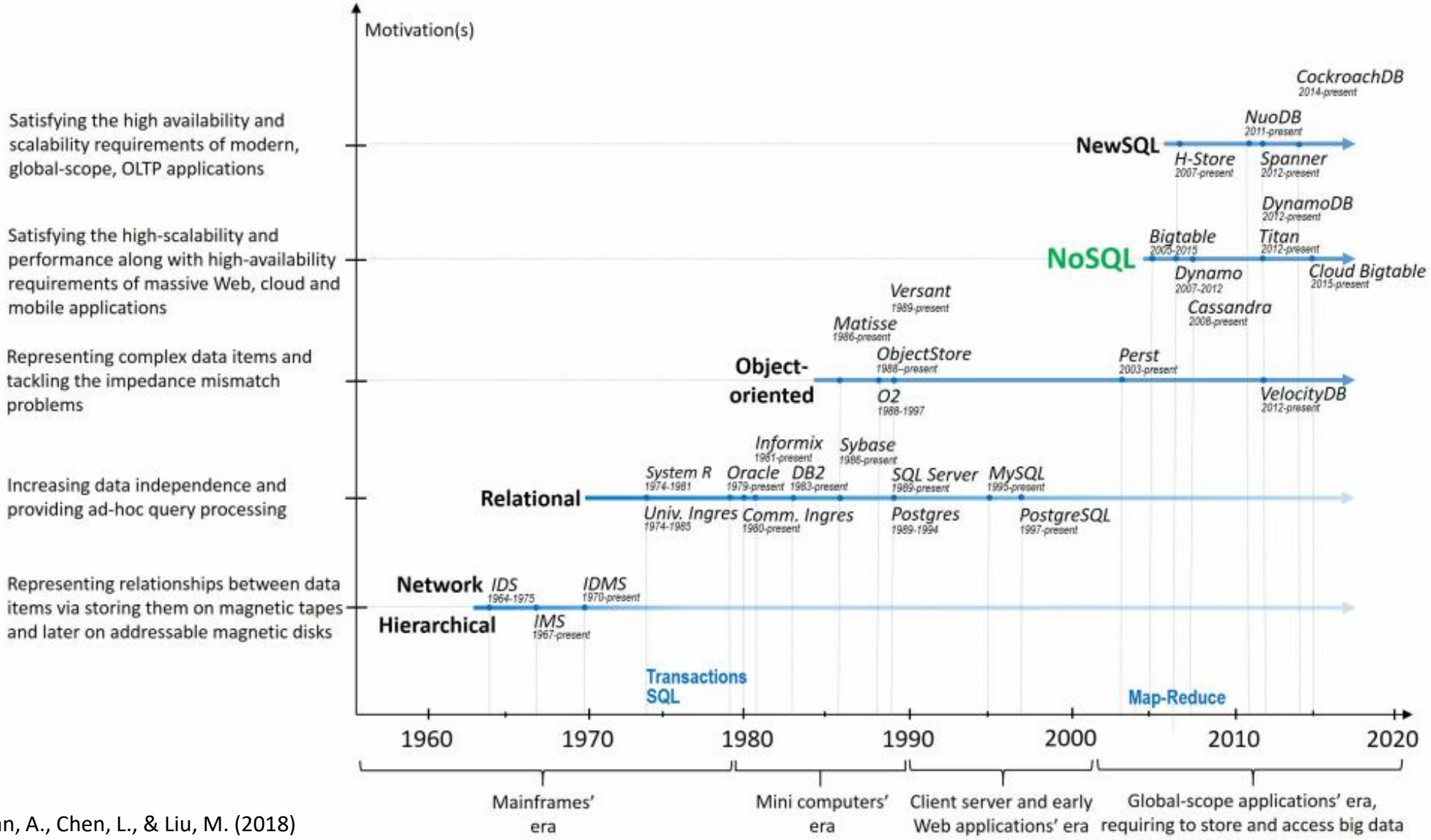


Introducción

- **¿Por qué SGBD?**
  - **Intercambio y reutilización de datos**
    - Consistencia, corrección, compacidad…
    - Concurrencia, aislamiento, transacciones…
  - **Interfaz y lenguajes unificados**
    - **Lenguaje de consulta** con operaciones de definición y manipulación de datos.
    - Herramientas para el desarrollo de aplicaciones.
  - **Seguridad de la información**
    - Autenticación de usuarios, autorización de acceso…
  - **Administración y mantenimiento**
    - Replicación, copias de seguridad, recuperación, migración, optimización…
  - Simplifica el **desarrollo de aplicaciones** caracterizadas por el uso **intensivo de DATOS**

Introducción

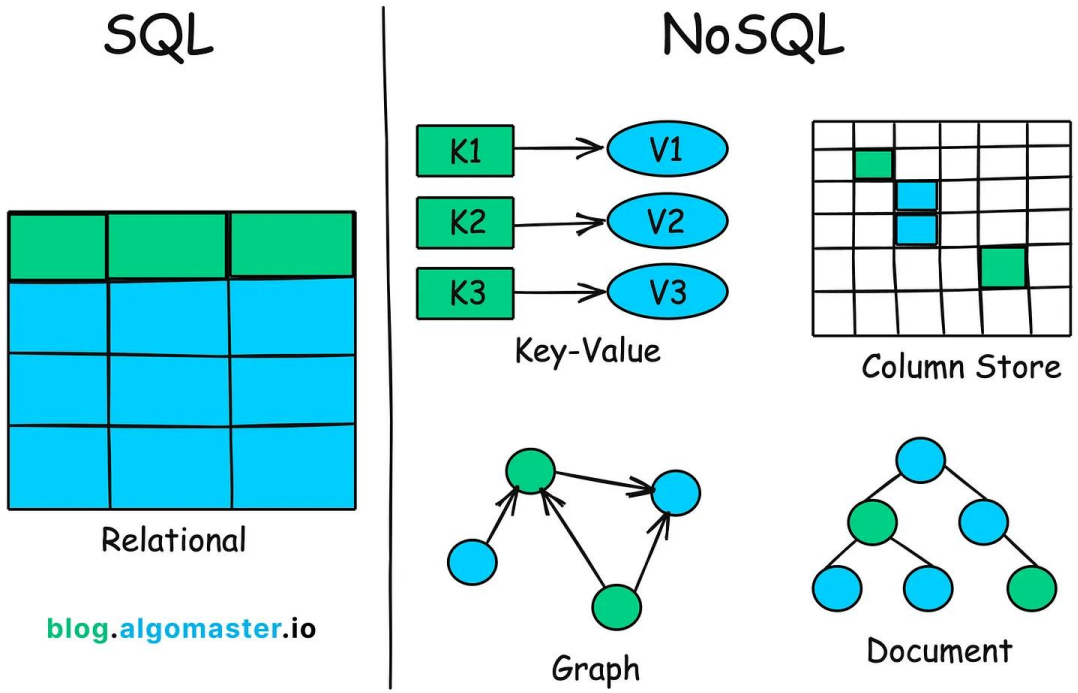
- **Historia**



Davoudian, A., Chen, L., & Liu, M. (2018)

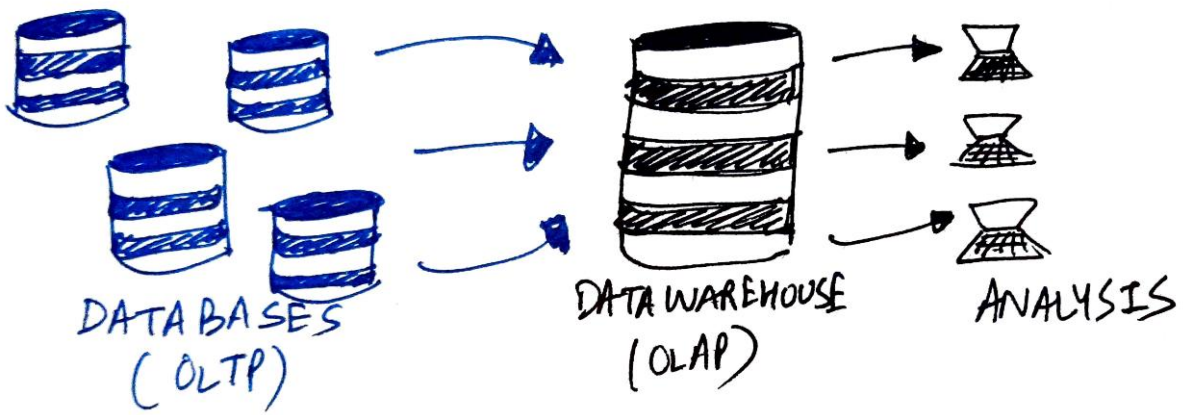
Introducción

- **Historia**
  - **Modelos y sistemas de bases de datos**
    - Bases de datos de **red** y **jerárquicas**
    - Bases de datos **relacionales (SQL)**
    - Bases de datos de **objetos** y objeto-relacionales
    - Bases de datos **XML, RDF, OWL**
    - Bases de datos **no relacionales** (**NoSQL)**
      - **clave-valor**, **documentos**, **grafos**, etc.
    - Bases de datos **New SQL**
    - **Flujo** (Stream), activas, deductivas, **espaciales**, **temporales**, probabilísticas, en tiempo real, en memoria, **embebidas**, etc.
  - Área en constante **evolución** con numerosos **desafíos**



Introducción

- **Historia**
  - **¿Por qué tantos sistemas de bases de datos diferentes?**
    - Diferentes **contextos**
      - **OLTP:** Pequeñas transacciones (**INSERT, UPDATE, DELETE**) realizadas en tiempo real y que ocurren de manera constante y rápida.
      - **OLAP: R**ecibe datos de OLTP para que pueda realizar **análisis**. Finalidad es **CONSULTA** (BI)
      - Computación en la nube, big data…
    - Diferentes **requisitos**
      - Rendimiento, escalabilidad, consistencia, disponibilidad…
    - Diferentes **arquitecturas**
      - Centralizadas, distribuidas, federadas..
    - Diferentes **formatos** de datos
      - Relaciones, objetos, grafos, árboles, …
      - Datos semiestructurados, no estructurados, textos, multimedia,…



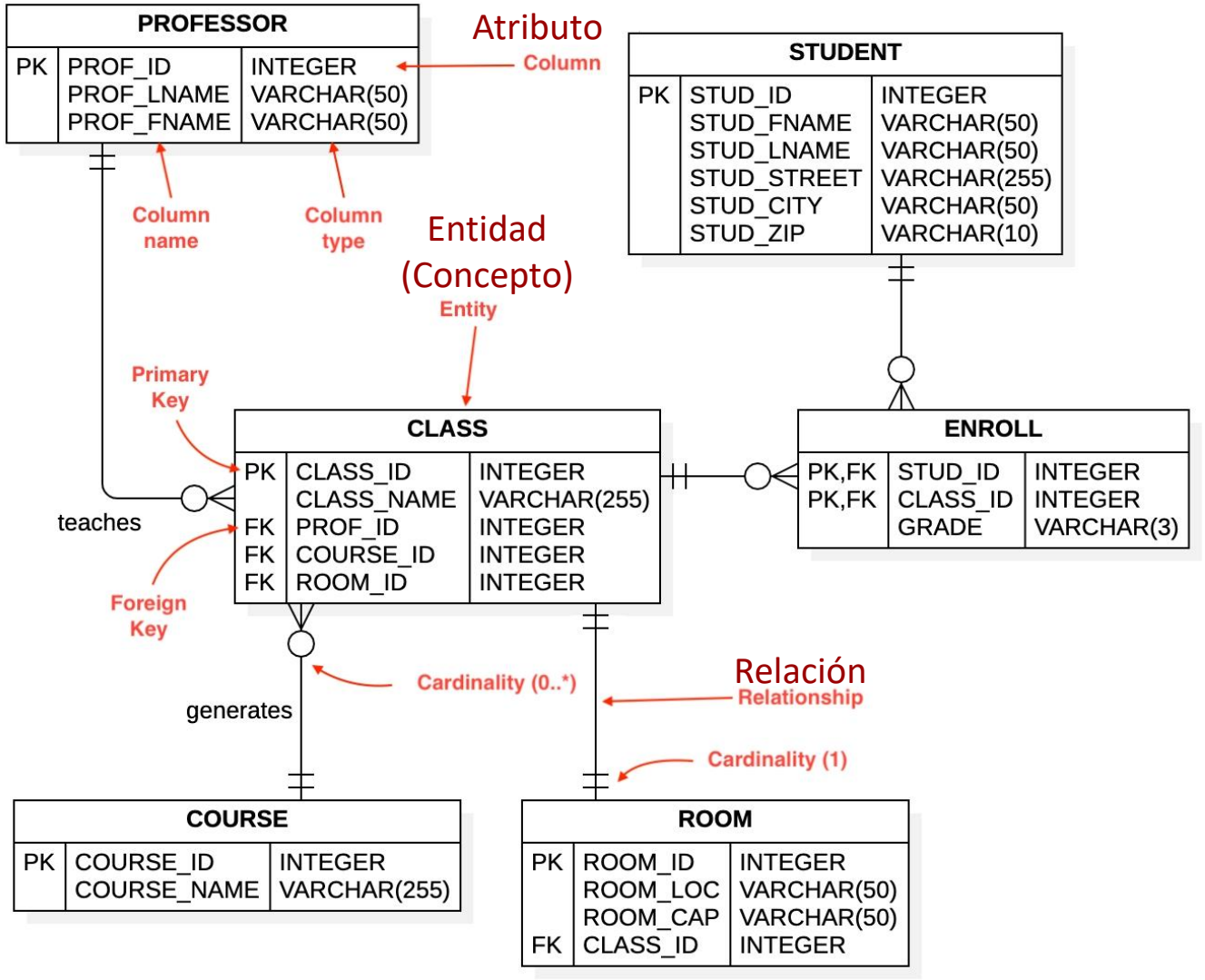
https://www.analyticsvidhya.com/blog/2020/11/oltp-vs-olap/

Bases de Datos

- **“Investigación y desarrollo de tecnología para dar soporte eficiente a la gestión de datos en los sistemas de información”**

Modelado de Bases de Datos

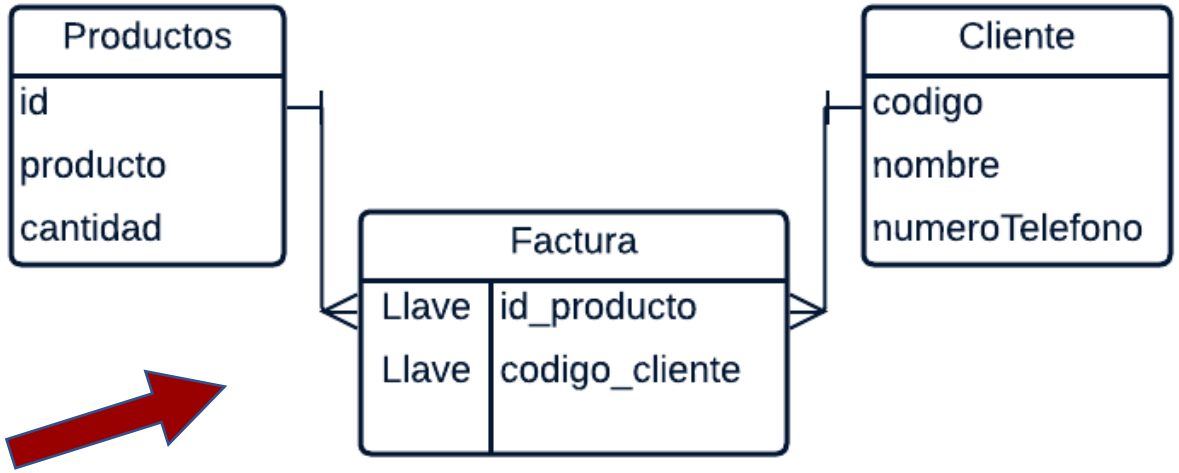
- **Modelo:** Conjunto de **<u>constructos</u>** para describir la **estructura** de una **BD**.
  - Estructura= {tipos de datos, atributos, conceptos, relaciones}
    - **Modelo Relacional (ER)** = {entidad, relación, atributos}
    - **Modelo de Objetos UML** = {clase, asociación, atributos}



https://docs.staruml.io/working-with-additional-diagrams/entity-relationship-diagram

Modelado de Bases de Datos

- **Esquema:** Descripción de una **BD** (proyecto), incluyendo los **conceptos** y las **relaciones** entre estos.
  - **Concepto**: Categoria genérica que representa una **entidade** o **cosa** sobre la cual se almazena datos.
  - **Atributos**: Características específicas de cada concepto (entidade).
  - **Esquema Relacional (ER)** = {Producto (id, producto, cantidad)}

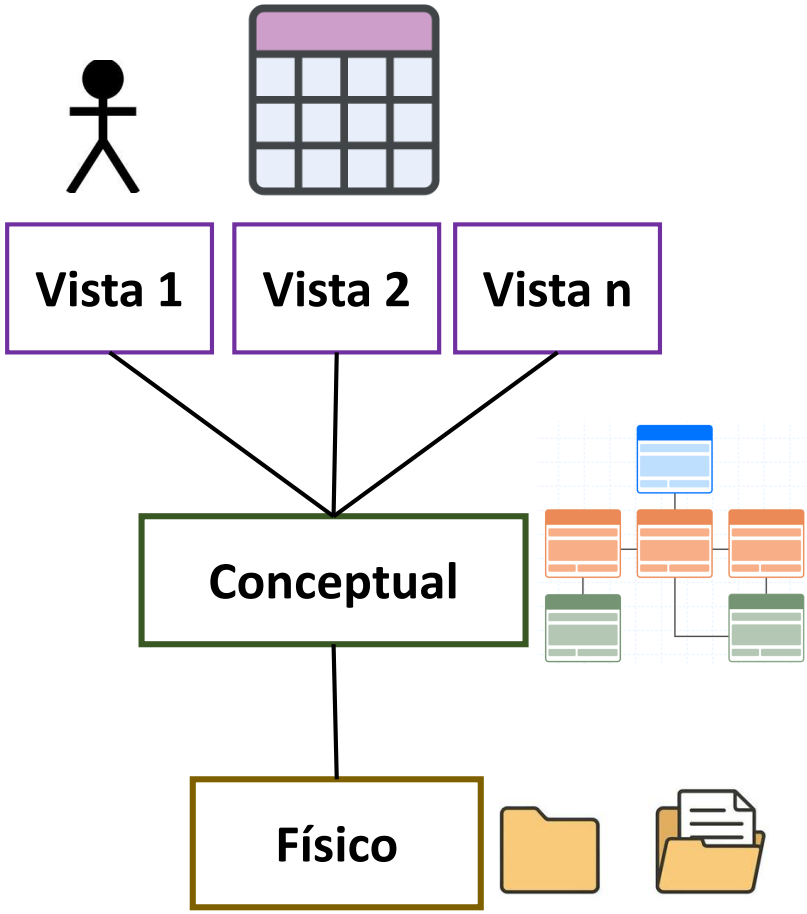


- **Diagrama:** Visualización del **esquema**
  - Diagrama ER
  - Diagrama de Clases UML
- **Instancia:** Información almacenada en la BD en un **momento dado**.

	A	B	B
1	ID	PRODUCTO	CANTIDAD
2	1	Libro	6
3	2	Agenda	1
4	3	Cuaderno	4

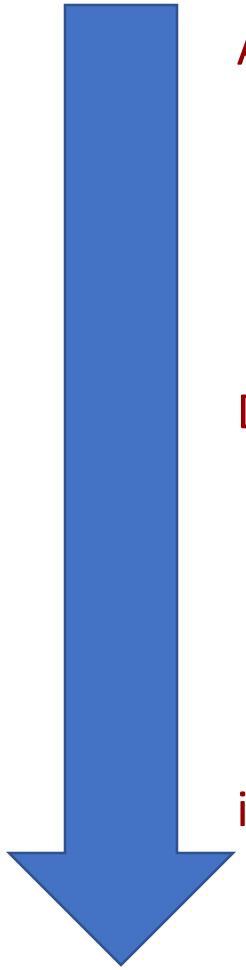
Modelado de Bases de Datos

- **Abstracción**
  - El SGBD debe proveer una **vista abstracta de los datos** para los usuarios
  - **Varias vistas**, un **único esquema conceptual (lógico)** y un **único esquema físico**.
    - Las **vistas** describen cómo los **usuarios ven los datos.**
    - El **esquema conceptual** define la **estructura lógica de los datos**.
    - El **esquema físico** describe los **archivos** e **índices** utilizados en las **búsquedas**.
  - Los **esquemas** se definen mediante **DDL (Lenguaje de Definición de Datos).**
  - Los **datos** se **modifican** y **consultan** mediante **DML (Lenguaje de Manipulación de Datos)**.



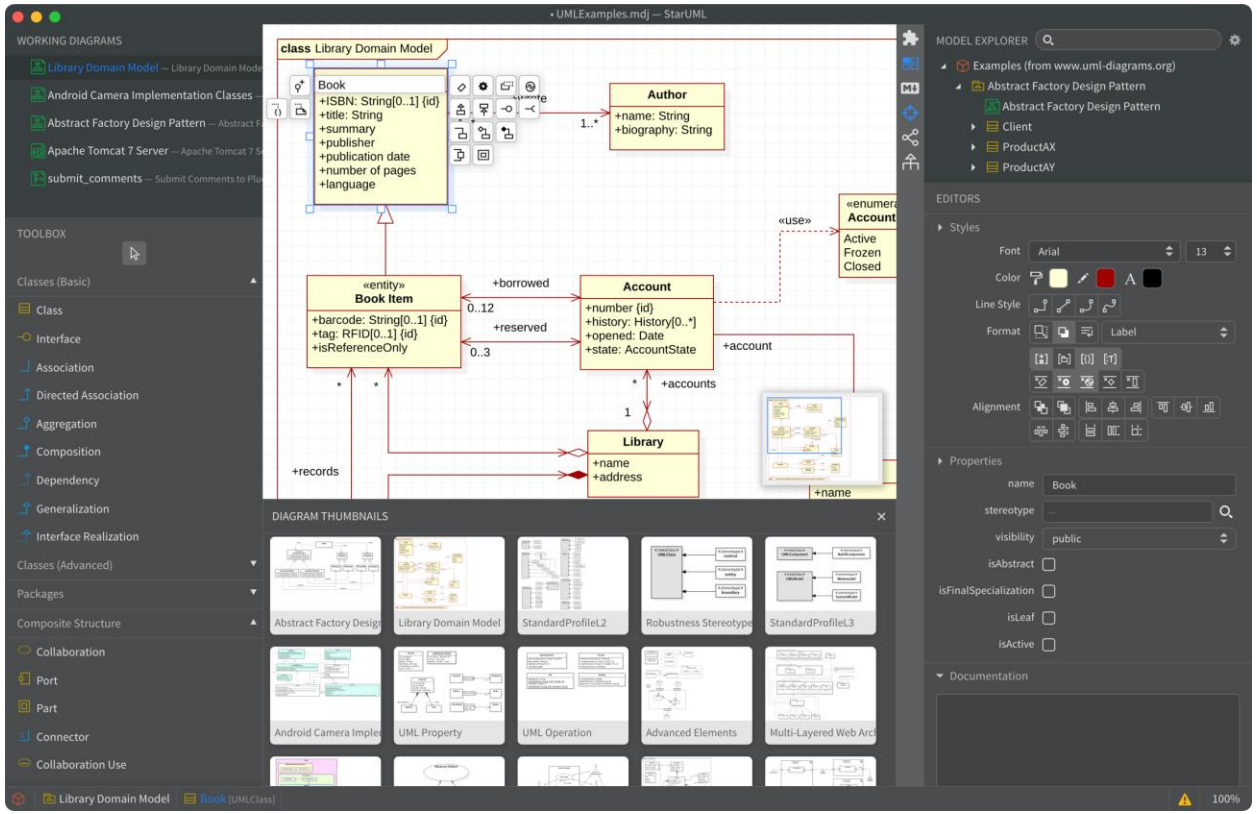
Modelado de Bases de Datos

- **Capas del Modelado**
  - **Capa conceptual**
    - **Modela** una **parte del <u>mundo real estructurado</u> relevante** para las aplicaciones creadas sobre nuestra base de datos. → **Modelo Conceptual**
  - **Capa lógica**
    - Especifica cómo se representan los **componentes conceptuales** (conceptos, relaciones y sus atributos) en **<u>estructuras de datos lógicas</u>** interpretables por máquinas. → **Modelo Lógico**
  - **Capa física**
    - Especifica cómo se implementan las **estructuras lógicas de bases de datos** en un entorno **técnico específico**. → **Modelo Físico**



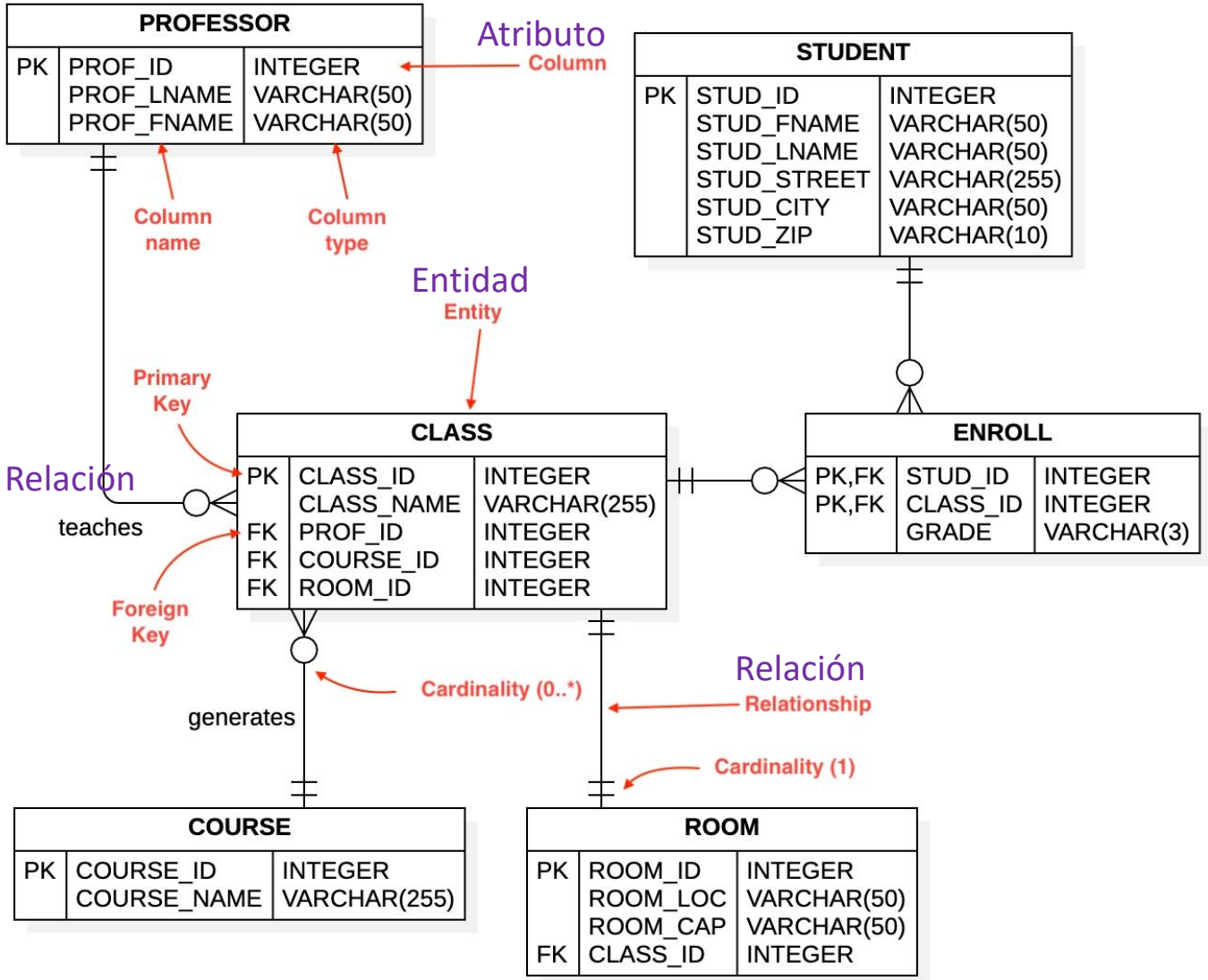
Modelado de Bases de Datos

- **Modelo Conceptual**
  - **Conceptos (entidades)** del mundo real y sus **relaciones**
  - **Lenguaje de modelado**:
    - **Diagrama Entidad-Relación (ER) = {entidades, relaciones, atributos}**
    - **Diagrama de Clases UML = {clases, asociaciones, atributos, operaciones}**
    - Lenguaje de Ontología Web (OWL)
    - Lógica de Predicados, …
  - Diversas **herramientas de modelado**, por ejemplo:
    - **draw.io**: https://app.diagrams.net/
    - **StarUML**: https://staruml.io/



Modelado de Bases de Datos

- **Modelo Conceptual**
  - **Diagrama Entidad-Relación (ER)**
    - **Notación:**
      - **Chen, Crow's foot, Bachman**
    - **Elementos clave:**
      - **Entidades:** Rectángulos (p.ej., Profesor, Alumno).
      - **Atributos**: propiedades de las entidades, p.ej., nombre.
      - **Relaciones**: Conexiones entre entidades, p.ej., enseña.

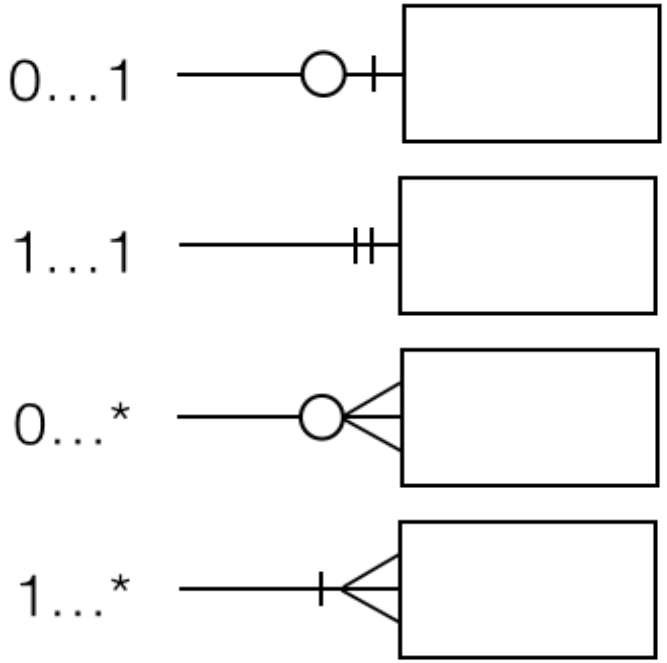


https://docs.staruml.io/working-with-additional-diagrams/entity-relationship-diagram

Modelado Relacional (SQL)

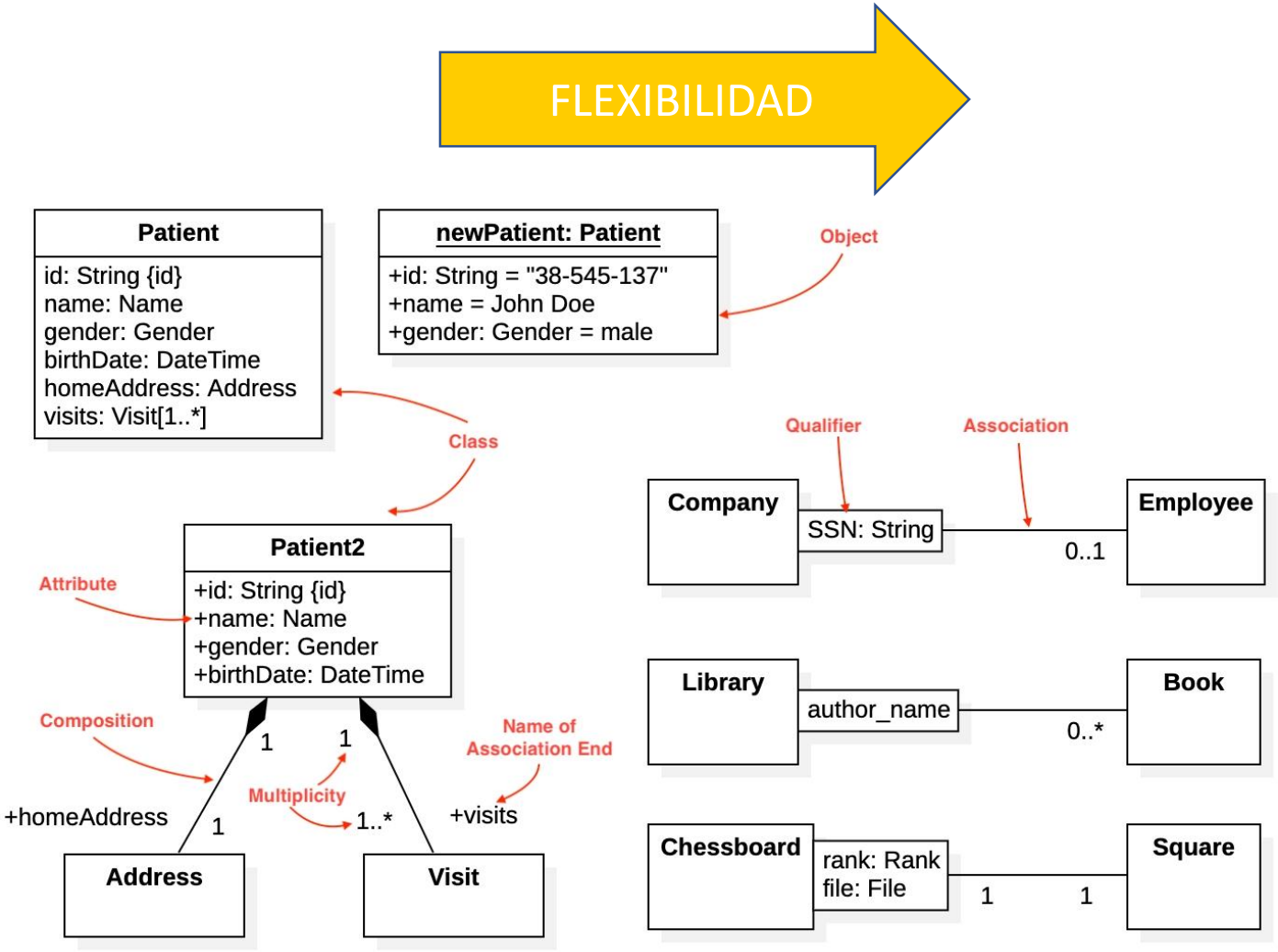
- **Modelo Conceptual**
  - **Diagrama Entidad-Relación (ER)**
    - **Elementos clave:**
      - **Cardinalidad: <u>número máximo de instancias</u>** de una **entidad B** con las que una **entidad A** pueden **asociarse** en un **momento dado**.
        - **0..1: ninguno o solo uno**
          - atributo
        - **1..1: solo uno**
          - atributo
        - **0..*: cero o muchos**
          - Adiciona una **columna** a la **tabla B** para **A**
        - **1..*: uno o muchos**
          - Adiciona una **columna** a la **tabla B** para **A**

DATOS Y RELACIONES



Modelado de Bases de Datos

- **Modelo Conceptual**
  - **Diagrama de Clases UML**
    - **Elementos clave:**
      - **Clase**: Nombre (negrita), Atributos (izq.), Operaciones (abajo).
      - **Atributos**: Propiedades de la clase (p.ej., nombre: String).
      - **Asociaciones**: Asociación, Agregación (parte-todo), Composición (fuerte parte-todo), Herencia (generalización/especialización , Realización (interfaces).
      - **Multiplicidad**: cardinalidad
      - ~~**Operaciones** (Métodos): Acciones que la clase puede realizar (p.ej., calcularTotal()).~~



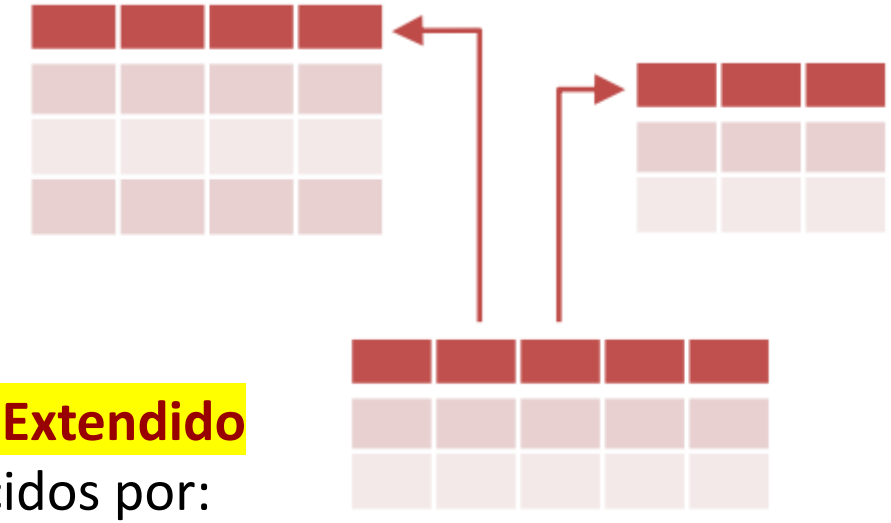
https://docs.staruml.io/working-with-uml-diagrams/class-diagram

Modelado de Bases de Datos

- **Modelo Lógico**
  - Modelo conceptual con más **detalles**: **tipos de datos de atributos**, mecanismos para implementar **relaciones** (p.ejm., claves, punteros, referencias,…) entre conceptos.
    - Adiciona detalles específicos de plataforma**.**
    - Modelo en lenguaje interpretable por maquina:
      - SQL, XML, OWL,
  - **Lenguaje de modelado**:
    - **Tablas (Relacional), Objetos, Grafos, Arboles**, **Colecciones**, Conjuntos, …
  - Diversas **herramientas de modelado**, por ejemplo:
    - **draw.io**: https://app.diagrams.net/
    - **StarUML**: https://staruml.io/

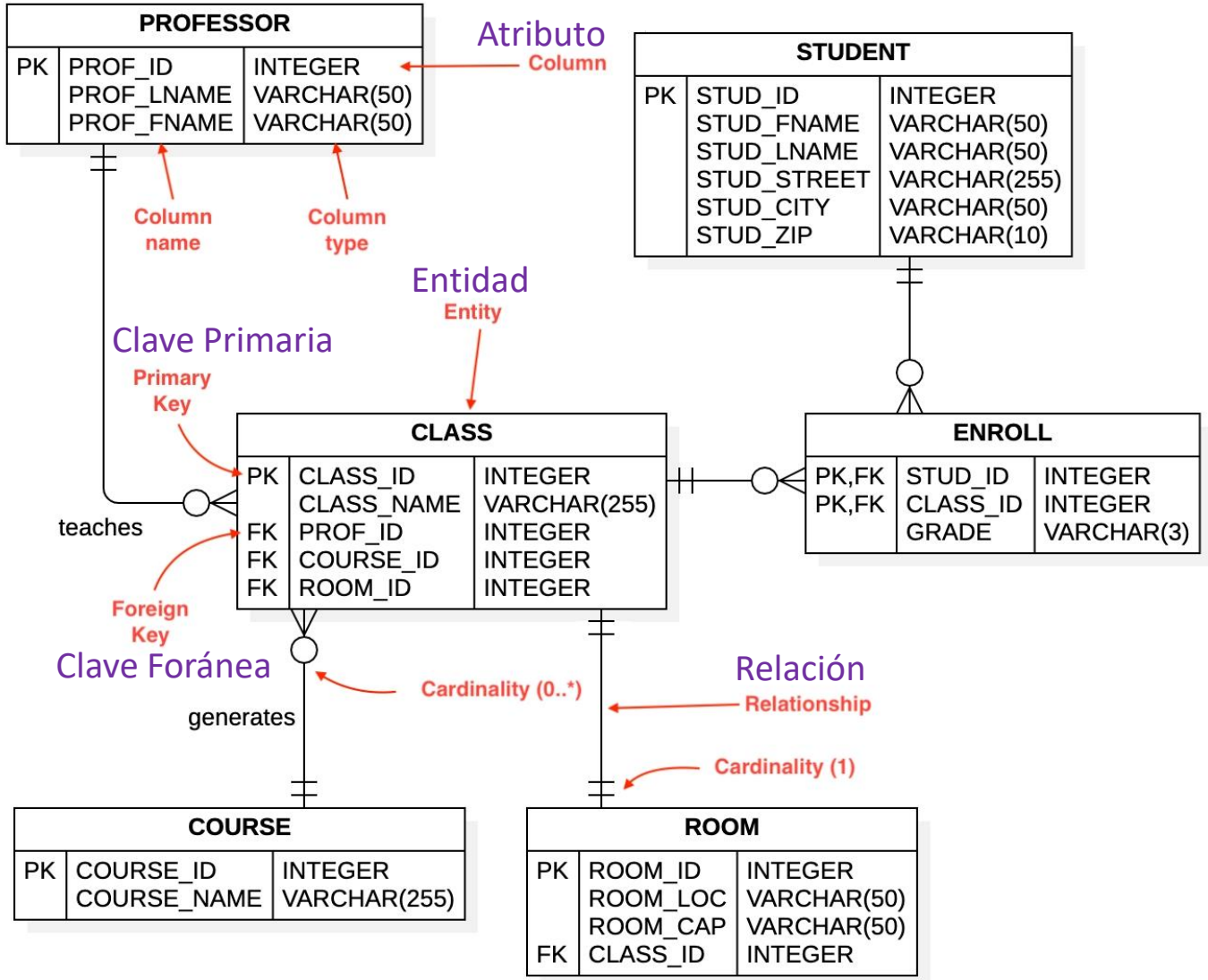
Modelado de Bases de Datos

- **Modelo Lógico**
  - Modelos basados en **Tablas (Modelo Relacional)**
    - **Estructura**
      - Organización de **datos** en **tablas (entidad - *relation*) bidimensionales**
        - **Columnas** para **atributos**
        - Cada **fila** representa un **registro** (**tupla**)
    - **Operaciones**
      - **Selección**, **proyección**, **unión**…
    - Ejemplos
      - **Modelo Relacional → Diagrama Entidad Relación Extendido**
      - … y diversos **modelos de tablas** derivados introducidos por:
        - **SQL** (tal como está estandarizado)
        - e implementaciones específicas como **Oracle**, **PostgreSQL**, **MySQL**…



Modelado de Bases de Datos

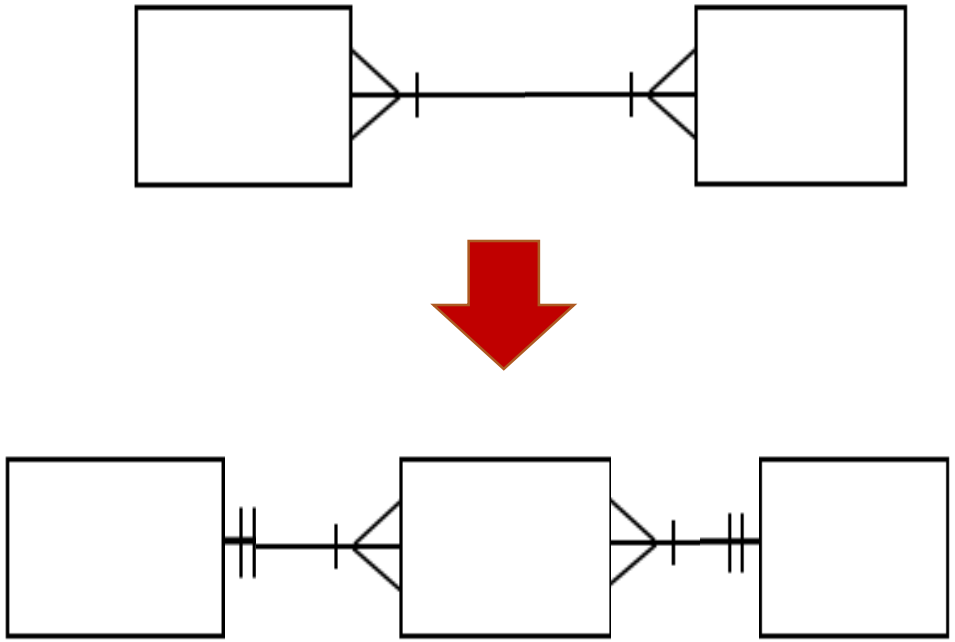
- **Modelo Lógico**
  - Modelos basados en **Tablas (Modelo Relacional)**
    - **Notación:**
      - **Chen, Crow's foot, Bachman**
    - **Elementos clave:**
      - **Clave primaria**: identificador único compuesto por uno o más atributos (p.ej., prof_id).
      - **Clave foránea**: Una entidad puede poseer un atributo que referencia a la clave primaria de otra entidad.
      - Una **relación** (***relationship***) vincula dos **entidades** por medio de sus **claves primaria y foránea.**



https://docs.staruml.io/working-with-additional-diagrams/entity-relationship-diagram

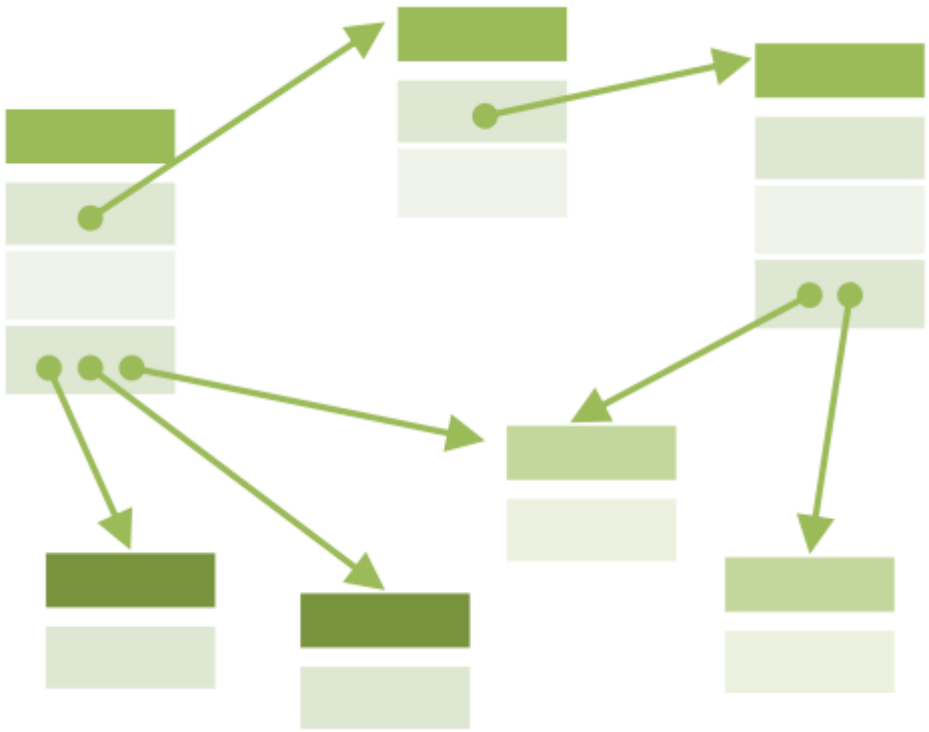
Modelado de Bases de Datos

- **Modelo Lógico**
  - Modelos basados en **Tablas (Modelo Relacional)**
    - **Notación:**
      - **Chen, Crow's foot, Bachman**
    - **Elementos clave:**
      - **Cardinalidad: <u>número máximo de instancias</u>** de una **entidad B** con las que una **entidad A** pueden **asociarse** en un **momento dado**.
        - …
        - ***..*: muchos a muchos**
          - Adiciona una **tabla C** entre **A** y **B**
          - **C(clave primaria de A, clave primaria de B** [, atributos de la relación]**)**



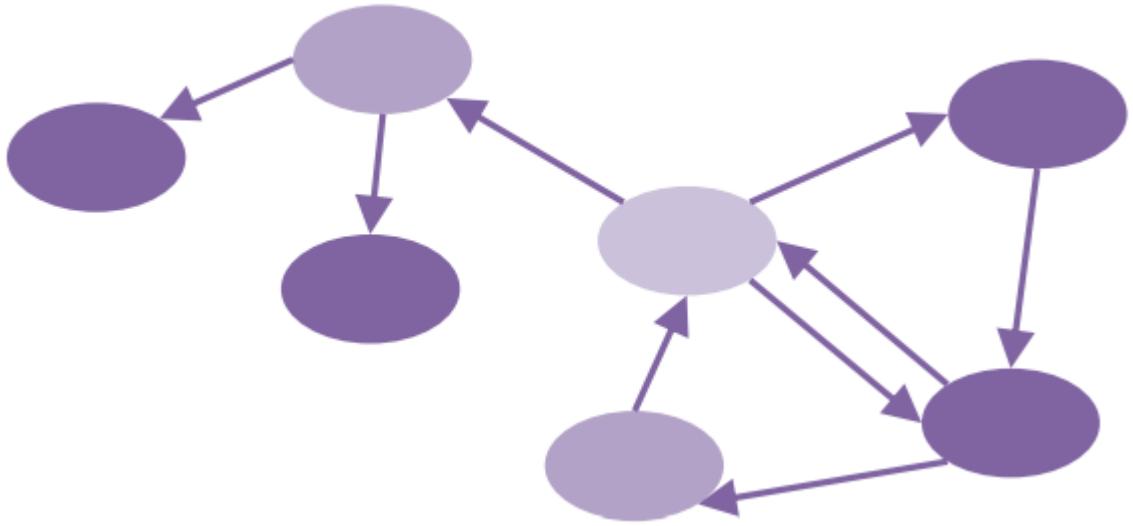
Modelado de Bases de Datos

- **Modelo Lógico**
  - Modelos basados en **Objetos**
    - **Estructura**
      - **Objetos** (conceptos) con **atributos**
      - **punteros** o **referencias** entre **objetos**
    - **Operaciones**
      - **navegación**
    - **Motivación**
      - **Análisis y Diseño Orientado a Objetos (OO)**
      - … y principios como:
        - **encapsulamiento**
        - **Herencia, …**
      - **Diagrama de Clases UML**



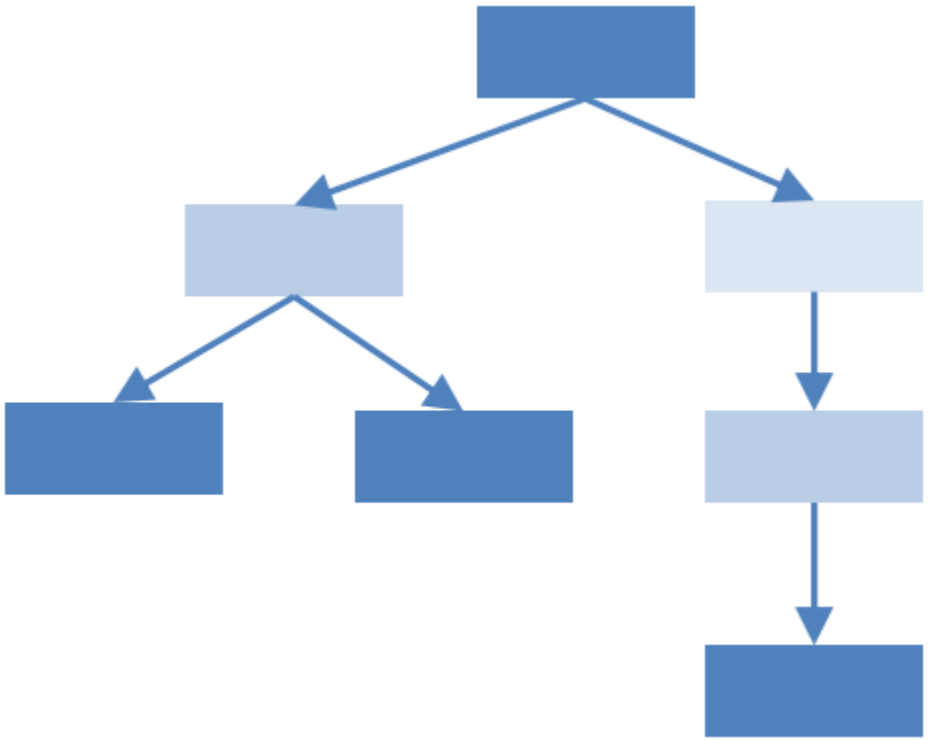
Modelado de Bases de Datos

- **Modelo Lógico**
  - Modelos basados en **Grafos**
    - **Estructura**
      - **Vértices** (conceptos), **aristas** y **atributos**
    - **Operaciones**
      - **Recorrido, algoritmos de grafos, *pattern matching***…
    - Ejemplos
      - **Modelo en red**
      - **Resource Description Framework (RDF)**
      - **Neo4j**, InfiniteGraph, OrientDB, FlockDB, …



Modelado de Bases de Datos

- **Modelo Lógico**
  - Modelos basados en **Arboles**
    - **Estructura**
      - **Vértices** (conceptos) con **atributos**
      - **Aristas** entre **vértices**
    - **Motivación**
      - **Jerarquías**, **categorías**, **Datos semiestructurados** …
    - Ejemplos
      - **Modelo jerárquico**
      - Documentos **XML**
      - Documentos **JSON**



Modelado de Bases de Datos

- **Modelo Lógico**
  - Diferentes modelos son adecuados para diferentes situaciones.
  - Depende de:
    - **Características de los datos**
      - Naturaleza real de los conceptos del mundo real y sus relaciones
    - **Capacidad de consulta**
      - Patrones de acceso disponibles, capacidad expresiva, etc.
    - **Uso previsto**
      - **Almacenamiento** (datos **JSON** en bases de datos **orientadas a documentos**, etc.)
      - **Intercambio** (documentos **XML** enviados por **servicios web**, etc.)
      - **Publicación** (**triples RDF** que forman la **Web de Datos**, etc.) en la web
    - **Requisitos de calidad:**
      - **Consistencia, escalabilidad, recuperación, …**

Modelado de Bases de Datos

Identificar ***stakeholders***, cualquier parte interesada y relevante para nuestra aplicación, por ejemplo, usuarios, inversores, propietarios, **expertos en el dominio**, etc.

Análisis de Requisitos

- Identificar **conceptos**
- Identificar tipos de **relaciones**
- Identificar **atributos**
- Identificar **restricciones (reglas de negocio)**

Modelado Conceptual

- Escoge un **lenguaje** de modelado
- Crear el **esquema conceptual**
- Crear el **diagrama** del Esquema

Diagrama Entidad-Relación (ER)
o Diagrama de Clases UML

Modelado Lógico

- Transforma el esquema conceptual a un **modelo lógico**
- Traduce modelo lógico a **instrucciones** de **creación** de **BD**

Modelo Relacional (SQL)
o No Relacional (NoSQL)

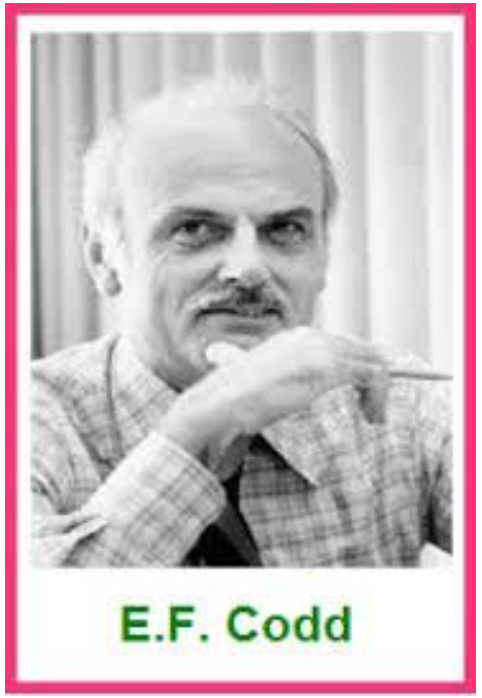
Modelado Físico

- Esquema **físico** en un SGBD

Adapte iterativamente su esquema a los requisitos que cambian con el tiempo.

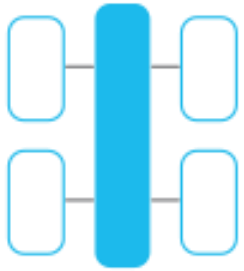
Modelo Relacional (SQL)

- Estándar para la gran mayoría de DBMS SQL Server, Oracle, PostgreSQL, MySQL, DB2, etc.
- Los **datos** se estructuran según el **modelo relacional**
- Elementos básicos: **relaciones** (tablas) y **registros** (tuplas)
- Características fundamentales
  - Restricciones de integridad (PK, FK, UK, CK, NN)
  - Normalización
  - SQL (lenguaje de consulta estructurado)



SQL Database

Relational



Analytical(OLAP)

Codd, E.F. (1970). "A Relational Model of Data for Large Shared Data Banks". Communications of the ACM 13 (6): 377–387. doi:10.1145/362384.362685

Modelo Relacional (SQL)

- Permite almacenar **entidades**, **relaciones** y sus **atributos** en **Tablas** (*relations*).
  - **Tabla** = conjunto de **filas**; cada **fila** representa una **entidad**; los **valores** de los **atributos** se almacenan en **columnas**.
  - Las **tablas** son más intuitivas, pero ocultan importantes fundamentos matemáticos.
  - Cada **entidad** debe ser siempre **identificable**.
    - mediante uno o más atributos → **Clave Primaria**.
  - Una **entidad** puede poseer un **atributo** que referencia (**Clave Foránea**) a la clave primaria de otra entidad.
  - Una **relación** (***relationship***) vincula dos **entidades** por medio de sus **claves primaria y foránea.**
    - → **consistencia de los datos → Integridad Referencial**

Clave foránea

numero	nombre	institucion
1	Equipo 1	1
2	Equipo 2	1
3	Equipo 3	2

Clave primaria

numero	nombre	url
1	UNA	www.a.edu
2	UNB	www.b.edu
3	UNC	www.c.edu

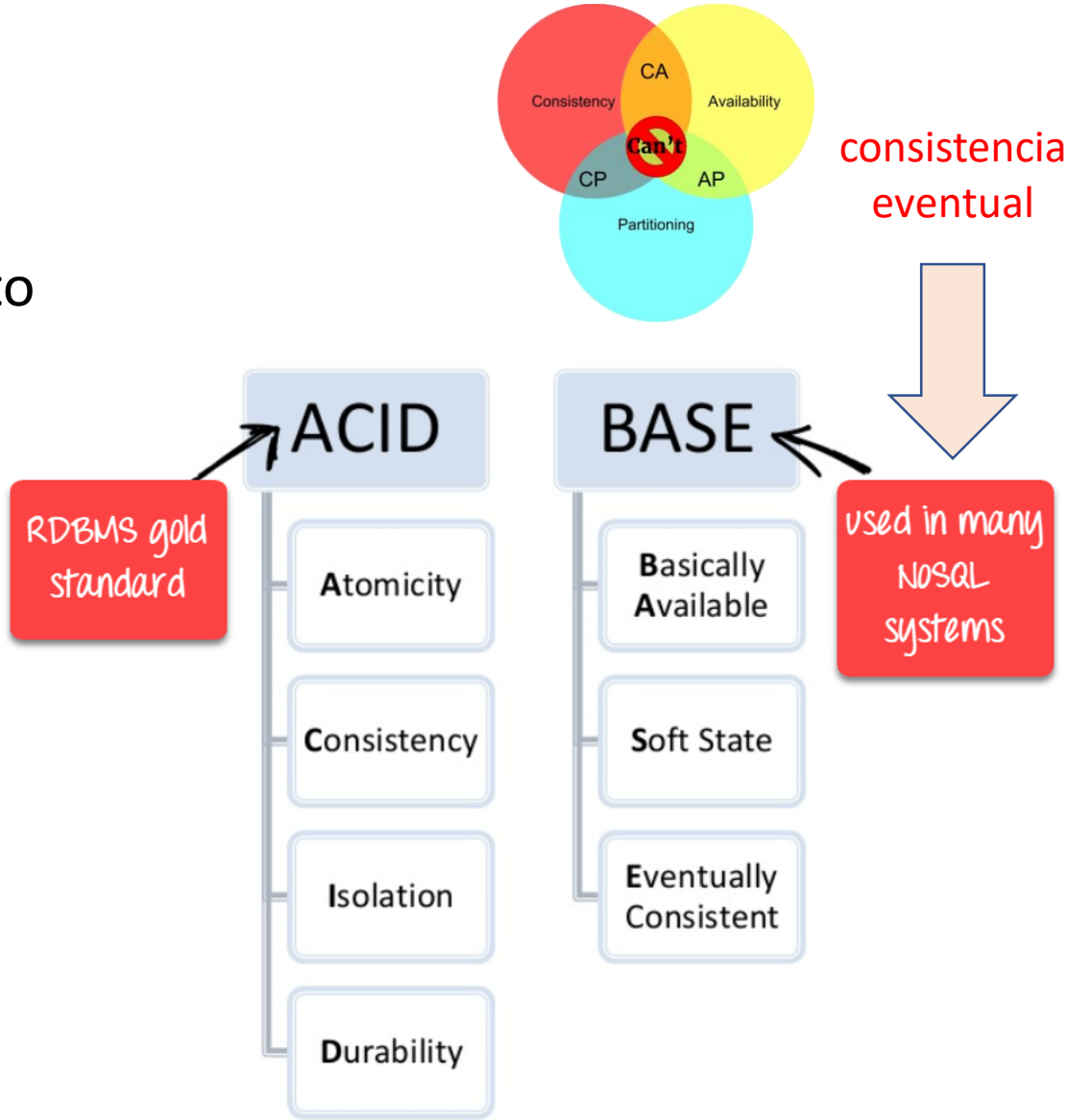
Clave primaria

Modelo Relacional (SQL)

- **Requisitos básicos (¿o consecuencias?)**
  - **Atomicidad** de los **atributos**
    - Solo se pueden usar tipos simples para dominios de atributos
  - **Unicidad** de las **tuplas**
    - La **tabla** es un **conjunto**, por lo que no pueden existir dos **tuplas idénticas**
  - **Orden indefinido**
    - La **tabla** es un conjunto, por lo que las **tuplas** no están mutuamente **ordenadas**
  - **Completitud** de los **valores**
    - No hay **huecos** en las **tuplas**; es decir, todos los **valores** están **especificados**
      - Sin embargo, se pueden agregar valores **NULL especiales** a los dominios de **atributos**

Modelo Relacional (SQL)

- **Propiedades Fundamentales ACID**
  - **A - Atomicidad**
    - Toda la transacción se ejecuta completamente o no se ejecuta en absoluto
  - **C - Consistencia**
    - La base de datos pasa de un estado válido a otro
  - **I - Aislamiento**
    - Las transacciones concurrentes no interfieren entre sí
  - **D - Durabilidad**
    - Los cambios realizados por una transacción son permanentes



Modelo Relacional (SQL)

- **Normalización: Proceso de simplificar datos a fin de eliminar información redundante**
  - **Particionar** una **tabla** en varias **tablas**
  - Establecer **relaciones** entre estas **particiones (clave primaria y foránea).**
  - Explore **1NF, 2NF, 3NF, y BCNF**

Timesheets				
Employee ID	First Name	Last Name	Ending Date	# of hours
1001	Anthony	Mann	2/6/99	40
1001	Anthony	Mann	2/13/99	45
1001	Anthony	Mann	2/20/99	50
1001	Anthony	Mann	2/27/99	55
1002	Alan	Ruth	2/6/99	0
1002	Alan	Ruth	2/13/99	20
1002	Alan	Ruth	2/20/99	40
1002	Alan	Ruth	2/27/99	45

Timesheets		
Employee ID	Week Ending Date	Weekly Time
1001	2/6/99	40
1001	2/13/99	45
1001	2/20/99	50
1001	2/27/99	55
1002	2/6/99	0
1002	2/13/99	20
1002	2/20/99	40
1002	2/27/99	45

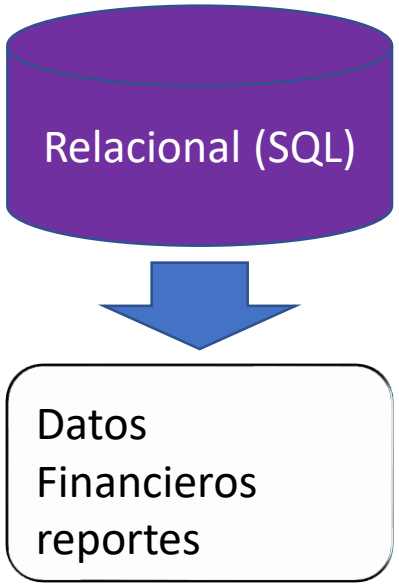
Employees		
Employee ID	First Name	Last Name
1001	Anthony	Mann
1002	Alan	Ruth

https://www.relationaldbdesign.com/sql-database-creation/module3/databaseNormalization-definition.php

Modelo Relacional (SQL)

- Adecuado cuando tus **datos son estructurados (e inmutables)** y las **relaciones** son **importantes (integridad)**.

	SQL
Modelo de almacenamiento	**Tablas** con **columnas** y **filas** fijas
Historia	Desarrollado en la década de 1970, con un enfoque en la reducción de datos duplicados
Ejemplos	Oracle, MySQL, Microsoft SQL Server y PostgreSQL
Esquemas	Rígidos
Escalabilidad	**Vertical** (con más potencia de procesamiento en la misma máquina)
Transacciones	Soporta
JOINS	Normalmente es necesario
El mapeo de datos a objetos	Requiere un **ORM (object-relational mapping)**



Modelado Relacional

1. Análisis de Requisitos

- Comenzar con los **requisitos** de las diferentes ***stakeholders***
  - Generalmente expresados en lenguaje natural
  - **Reuniones**, **debates**, **consultas**, etc.
- Identificar **aspectos importantes**:
  - tipos de **<u>conceptos</u>** del **mundo real**,
    - sus **características → <u>atributos</u>**,
  - **tipos de <u>relaciones</u>** entre **ellas**, y
    - sus **características → <u>atributos</u>,**
  - **restricciones** o **reglas de negocio (o condiciones)** aplicables **a conceptos y relaciones**
- … y abordar las **ambigüedades**

Ejemplo:

Nuestro entorno está formado por personas que pueden tener a otras como colegas. Una persona también puede ser miembro de varios equipos de investigación y trabajar en diversos proyectos de investigación. Un equipo está formado por personas que cooperan entre sí. Cada equipo tiene un líder, que debe ser profesor académico (auxiliar, asociado o titular). Un equipo actúa como una entidad individual que puede cooperar con otros equipos. Por lo general, forma parte formalmente de una institución oficial, por ejemplo, un departamento universitario. Un proyecto está formado por personas que trabajan en un proyecto, pero solo como miembros del equipo de investigación.

Modelado Relacional

1. Análisis de Requisitos

- Tipos de **conceptos** identificados
  - **Persona**
  - **Equipo**
  - **Proyecto**
  - **Profesor**
    - **Profesor Adjunto**
    - **Profesor Asociado**
    - **Profesor Titular**
  - **Institución**
  - **Departamento**

Ejemplo:

Nuestro entorno está formado por **personas** que pueden tener a otras como colegas. Una persona también puede ser **miembro** de varios **equipos** de investigación y trabajar en diversos **proyectos** de investigación. Un equipo está formado por personas que cooperan entre sí. Cada equipo tiene un líder, que debe ser **profesor** académico (**auxiliar**, **asociado** o **titular**). Un equipo actúa como una entidad individual que puede cooperar con otros equipos. Por lo general, forma parte formalmente de una **institución** oficial, por ejemplo, un **departamento** universitario. Un proyecto está formado por personas que trabajan en un proyecto, pero solo como miembros del equipo de investigación.

Modelado Relacional

1. Análisis de Requisitos

- Tipos de **relaciones** identificados
  - persona **es colega** de persona
  - persona **es miembro de** equipo
  - persona **trabaja en** proyecto
  - equipo **está formado** por personas
  - equipo **tiene como líder** a profesor
  - equipo **coopera con** equipos
  - equipo **forma parte** de institución
  - proyecto está **formado por** personas que es **miembro de** equipo

Ejemplo:

Nuestro entorno está formado por **personas** que **pueden tener** a otras **como colegas**. Una **persona** también **puede ser miembro** de varios **equipos** de investigación y **trabajar en** diversos **proyectos** de investigación. Un **equipo está formado** por **personas** que cooperan entre sí. Cada **equipo tiene un líder**, que debe ser **profesor** académico (**auxiliar**, **asociado** o **titular**). Un **equipo** actúa como una entidad individual que puede **cooperar con** otros **equipos**. Por lo general, **forma parte** formalmente de una **institución** oficial, por ejemplo, un **departamento** universitario. Un **proyecto** está **formado por** personas que **trabajan en** un **proyecto**, pero solo como **miembros del equipo** de investigación.

Modelado Relacional

1. Análisis de Requisitos

- Tipos de **relaciones** identificados
  - **Cardinalidad** (multiplicidad en UML)
    - Define **cuántas instancias** de un **concepto A** pueden **asociarse** con una instancia de un **concepto B** en un **momento dado**.
      - 1..1: Solo uno
      - 0..*: Cero o más
      - 1..*: uno o más
      - 0..1: Ninguno o solo uno

Ejemplo:

Nuestro entorno está formado por **personas** que **pueden tener** a otras **como colegas**. Una **persona** también **puede ser miembro** de varios **equipos** de investigación y **trabajar en** diversos **proyectos** de investigación. Un **equipo está formado** por **personas** que cooperan entre sí. Cada **equipo tiene un líder**, que debe ser **profesor** académico (**auxiliar**, **asociado** o **titular**). Un **equipo** actúa como una entidad individual que puede **cooperar con** otros **equipos**. Por lo general, **forma parte** formalmente de una **institución** oficial, por ejemplo, un **departamento** universitario. Un **proyecto** está **formado por** personas que **trabajan en** un **proyecto**, pero solo como **miembros del equipo** de investigación.

Modelado Relacional

1. Análisis de Requisitos

- Tipos de **relaciones** identificados
  - **Cardinalidad** (multiplicidad en UML)

0..* • persona **es colega** de persona

0..* • persona **es miembro de** equipo

0..* • persona **trabaja en** proyecto

1..* • equipo **está formado** por personas

1..1 • equipo **tiene como líder** a profesor

0..* • equipo **coopera con** equipos

1..1 • equipo **forma parte** de institución

1..1 • proyecto está **formado por** personas que es **miembro de** equipo

Ejemplo:

Nuestro entorno está formado por **personas** que **pueden tener** a otras **como colegas**. Una **persona** también **puede ser miembro** de varios **equipos** de investigación y **trabajar en** diversos **proyectos** de investigación. Un **equipo está formado** por **personas** que cooperan entre sí. Cada **equipo tiene un líder**, que debe ser **profesor** académico (**auxiliar**, **asociado** o **titular**). Un **equipo** actúa como una entidad individual que puede **cooperar con** otros **equipos**. Por lo general, **forma parte** formalmente de una **institución** oficial, por ejemplo, un **departamento** universitario. Un **proyecto** está **formado por** personas que **trabajan** en un **proyecto**, pero solo como **miembros del equipo** de investigación.

Modelado Relacional

1. **Análisis de Requisitos**
   - **Atributos** de **concepto** identificados
     - **Persona**
       - **Número personal**
       - **Nombre**
       - Uno o más **números de teléfono**
       - **Correo electrónico**
   - **Atributos** de **relación** identificados
     - **Membresía (miembro de)**
       - **de**
       - **hasta**

Ejemplo:

Cada persona tiene un **nombre** y se identifica con un **número personal**. Se puede llamar a una persona a su **número de teléfono**. Necesitamos saber al menos un número de teléfono. También necesitamos enviarle **correos electrónicos**.

Necesitamos saber **cuándo** una persona **se hizo** miembro de un proyecto y **cuándo terminó** su **membresía**.

Modelado Relacional

1. Análisis de Requisitos

- **Restricciones** o reglas de negocio identificadas
  - Un **proyecto** está **formado por** personas que **trabajan** en un **proyecto**, **pero solo** como **miembros** del **equipo** de investigación

Es común que un Modelo Conceptual no alcance a representar exactamente la realidad planteada

- Pueden ser expresado informalmente en **lenguaje natural (una nota)**

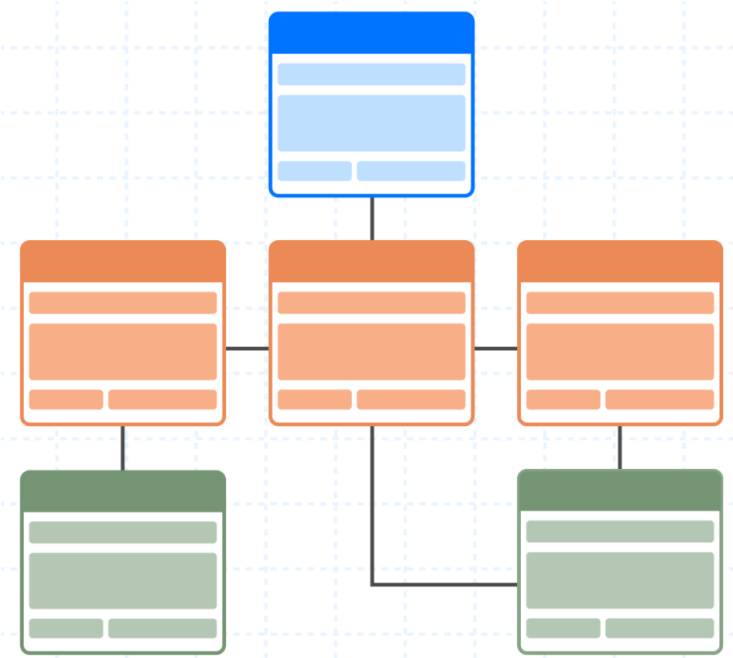
Ejemplo:

Nuestro entorno está formado por **personas** que **pueden tener** a otras **como colegas**. Una **persona** también **puede ser miembro** de varios **equipos** de investigación y **trabajar en** diversos **proyectos** de investigación. Un **equipo está formado** por **personas** que cooperan entre sí. Cada **equipo tiene un líder**, que debe ser **profesor** académico (**auxiliar**, **asociado** o **titular**). Un **equipo** actúa como una entidad individual que puede **cooperar con** otros **equipos**. Por lo general, **forma parte** formalmente de una **institución** oficial, por ejemplo, un **departamento** universitario. Un **proyecto** está **formado por** personas que **trabajan** en un **proyecto**, **pero solo** como **miembros** del **equipo** de investigación.

Modelado Relacional

2. Modelado Conceptual

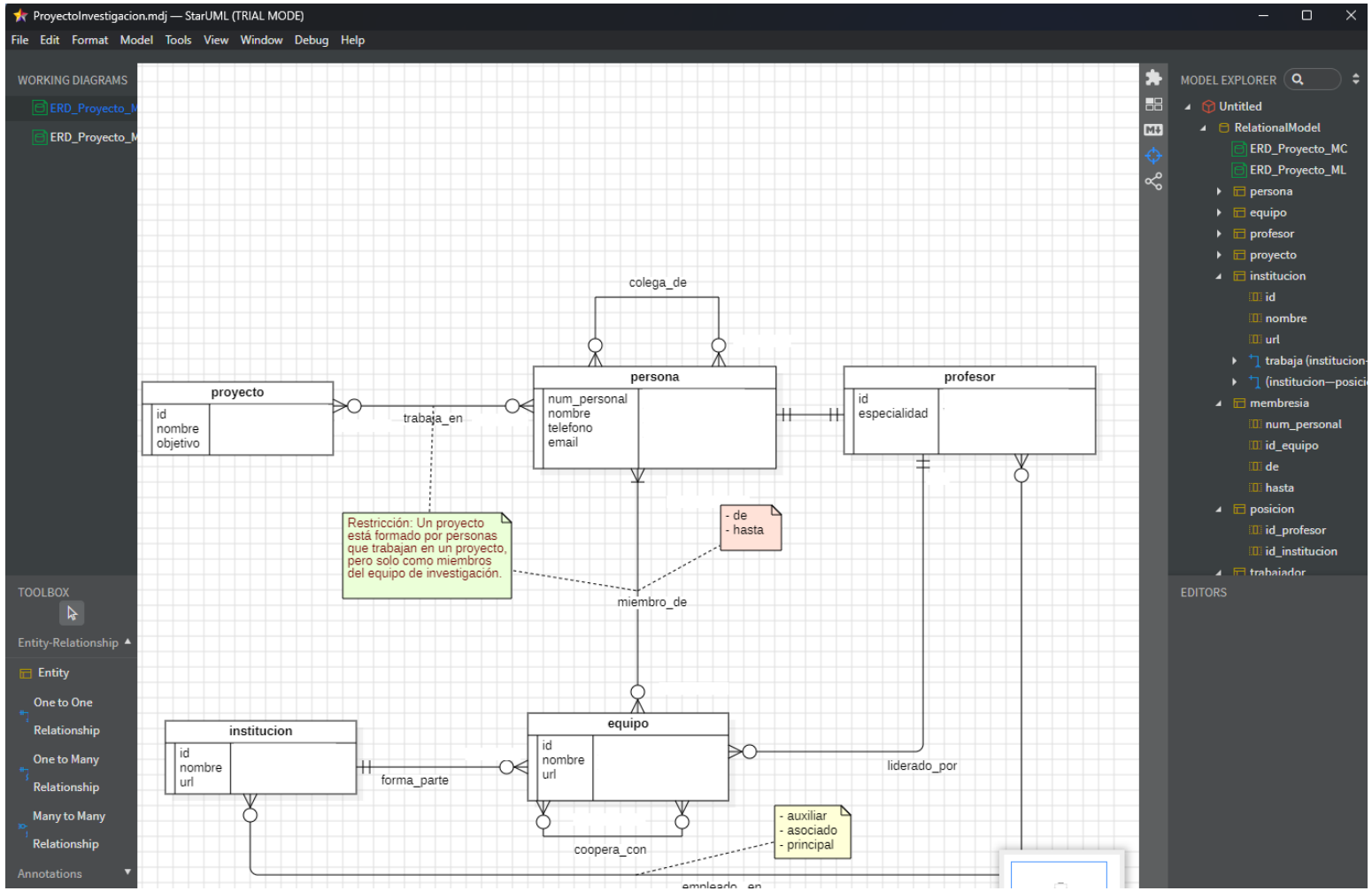
- Escoger un **lenguaje de modelado**:
  - **Diagrama Entidad-Relación (ER) = {entidades, relaciones, atributos, cardinalidad, restricciones}**
- Modelar el **esquema conceptual** (conceptos, atributos, relaciones y restricciones) utilizando los **constructos** proporcionados por el **lenguaje de modelado** elegido:
- **Esquema conceptual → diagrama**.



Modelado Relacional

2. Modelado Conceptual

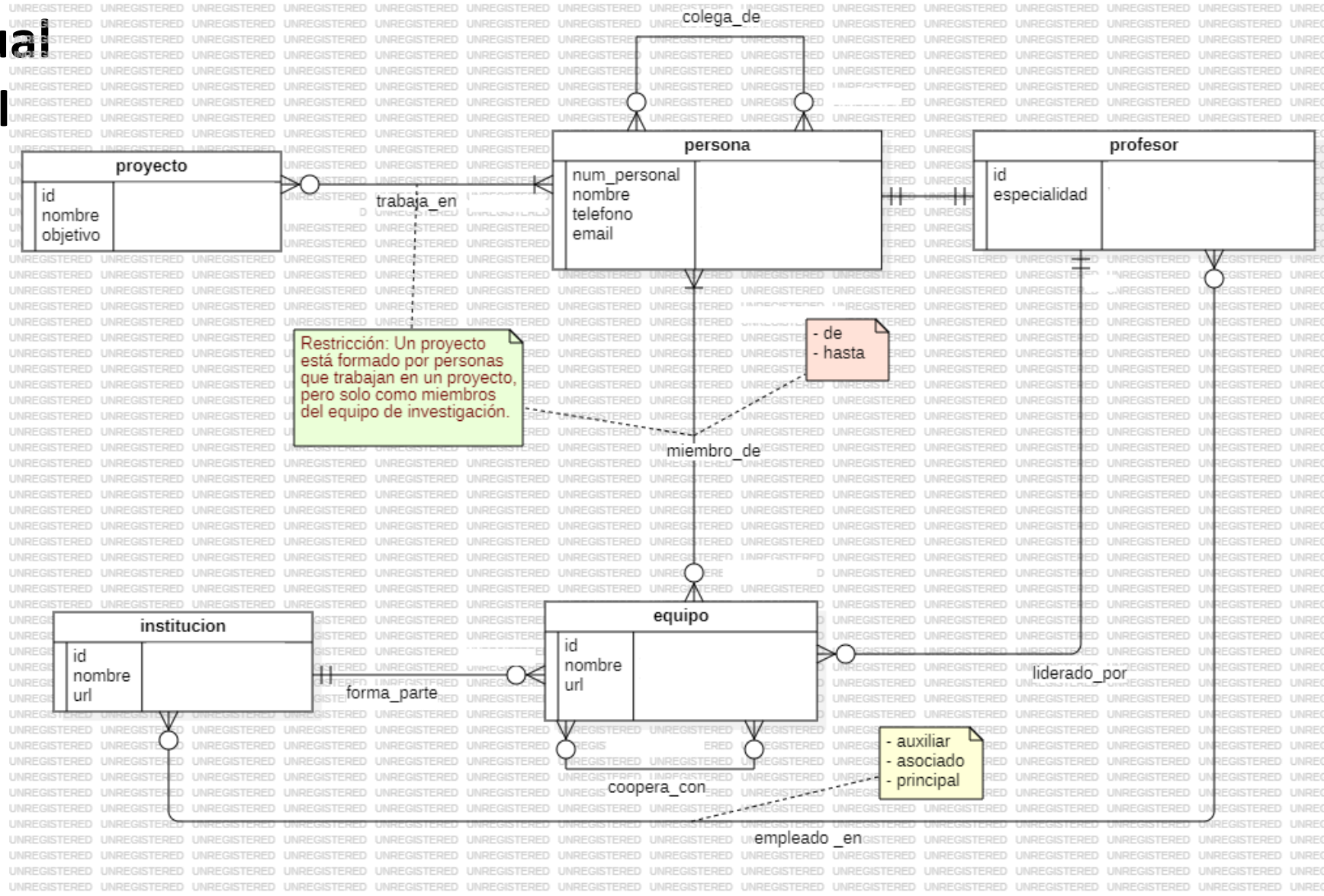
- **StarUML**: https://staruml.io/



Modelado Relacional

2. Modelado Conceptual

- **Esquema conceptual**



Modelado Relacional

3. Modelado Lógico

- Escoger un **lenguaje de modelado lógico**:
  - **Tablas (Relacional)**
- Especificar los componentes **conceptuales** (conceptos, atributos y relaciones) en las **estructuras lógicas** de **bases de datos** utilizando los constructos proporcionados por el lenguaje de modelado elegido:
  - Normalmente, un **esquema** **conceptual más** **detallado**.
- Traducir el **esquema lógico** a **instrucciones** que deben ser ejecutadas para **crear la BD en un SGBD**.
  - Modelo Relacional (ER) → Instrucciones en ***Structured query language (SQL)***
    - **DDL** (lenguaje de definición de datos)

Modelado Relacional

3. Modelado Lógico

- **Modelo Relacional (Tablas)**
  - **equipo**

Clave foránea

numero	nombre	url	institucion
1	Equipo 1	www.x.com	1
2	Equipo 2	www.y.com	1
3	Equipo 3	www.z.com	2

atributo

Registro (datos)

Clave primaria

  - **institución**

numero	nombre	url
1	UNA	www.a.edu
2	UNB	www.b.edu
3	UNC	www.c.edu

Clave primaria

Modelado Relacional

3. Modelado Lógico

- **Modelo Relacional (Tablas)**
  - **equipo**

Clave foránea

numero	nombre	url	institucion
1	Equipo 1	www.x.com	1
2	Equipo 2	www.y.com	1
3	Equipo 3	www.z.com	2

Clave primaria

  - **institución**

numero	nombre	url
1	UNA	www.a.edu
2	UNB	www.b.edu
3	UNC	www.c.edu

Clave primaria

Relación (*relationship*) con integridad referencial

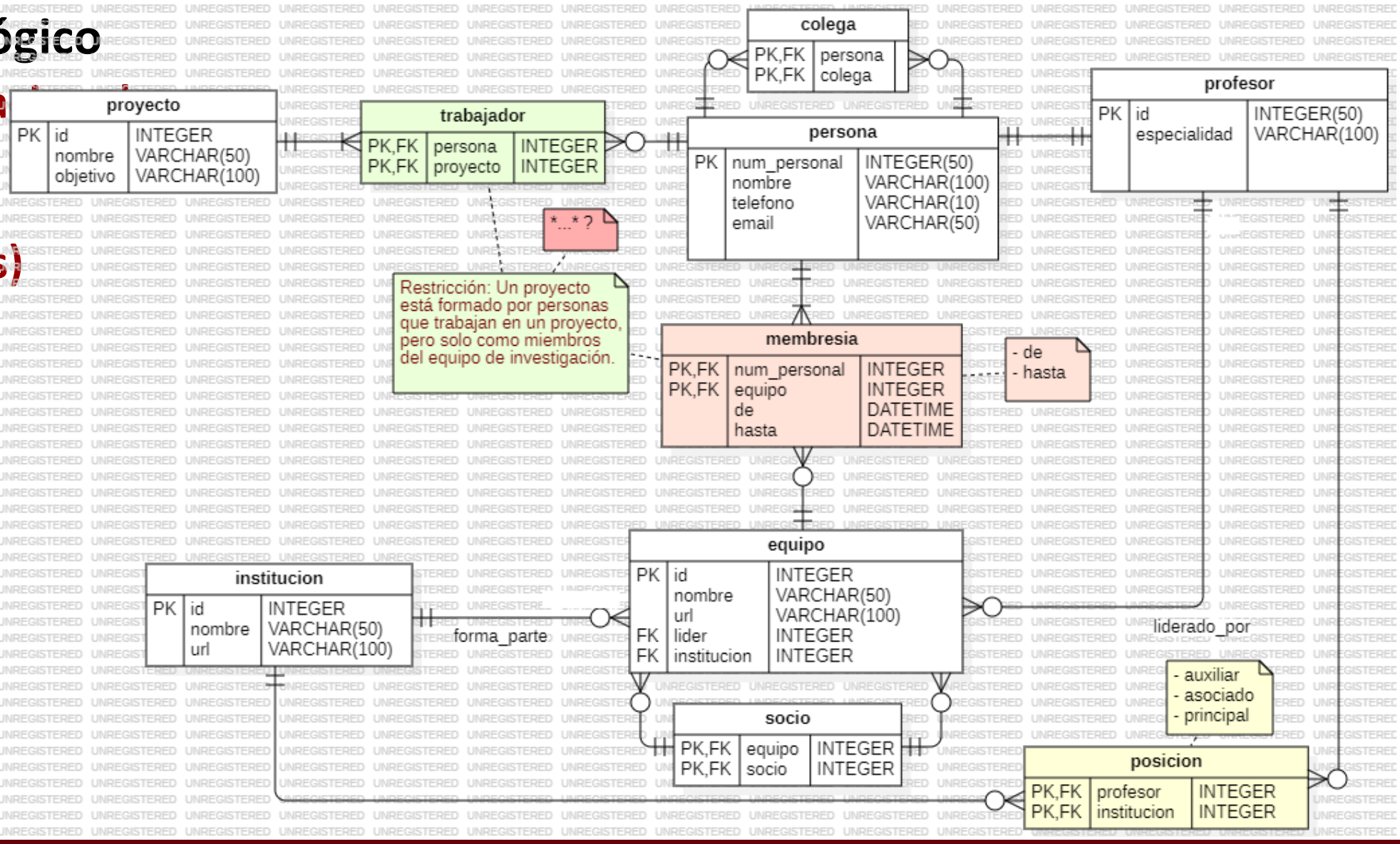
Una **relación** (***relationship***) vincula dos **tablas** por medio de sus **claves primaria y foránea.**

Garantiza la **consistencia de los datos (integridad referencial)**

Modelado Relacional

3. Modelado Lógico

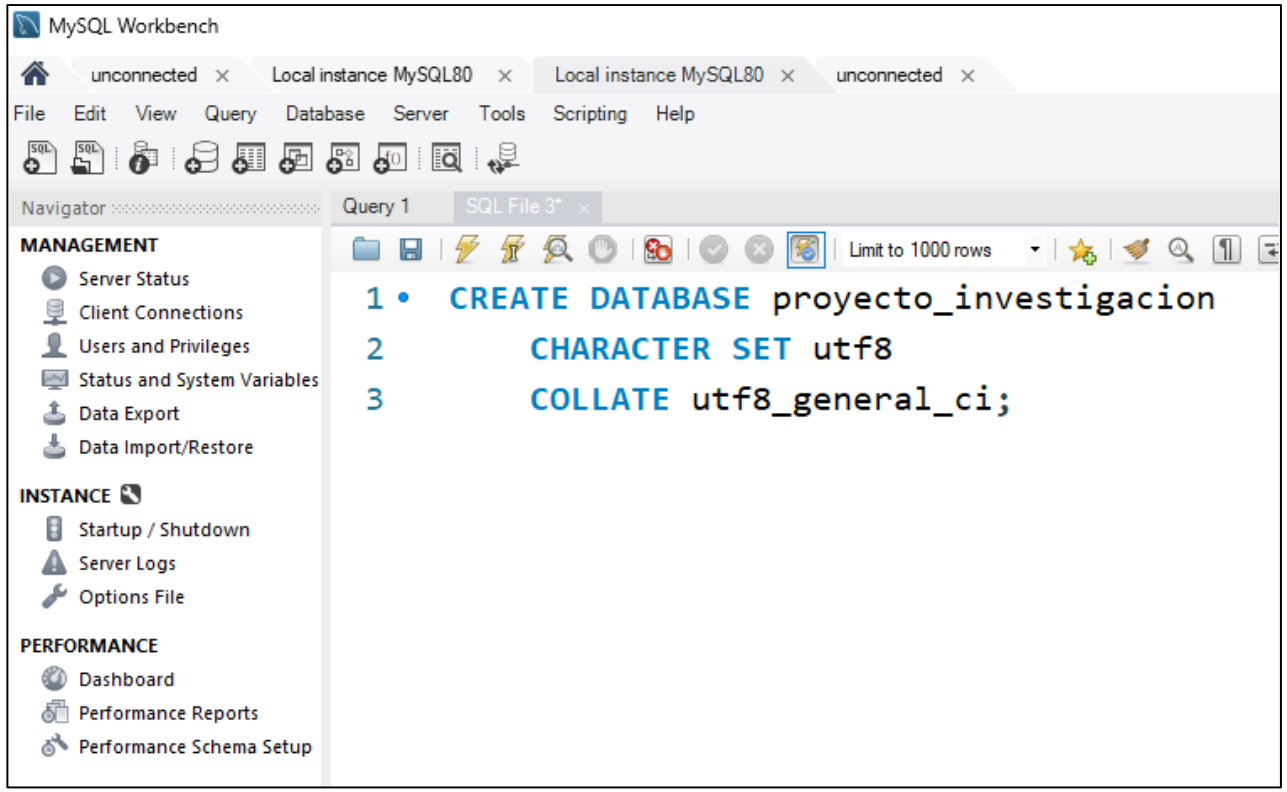
- **Modelo Relacional (tablas, relaciones y restricciones)**



Modelado Relacional

3. Modelado Lógico

- Modelo Relacional (ER) → Instrucciones en ***Structured query language (SQL)***

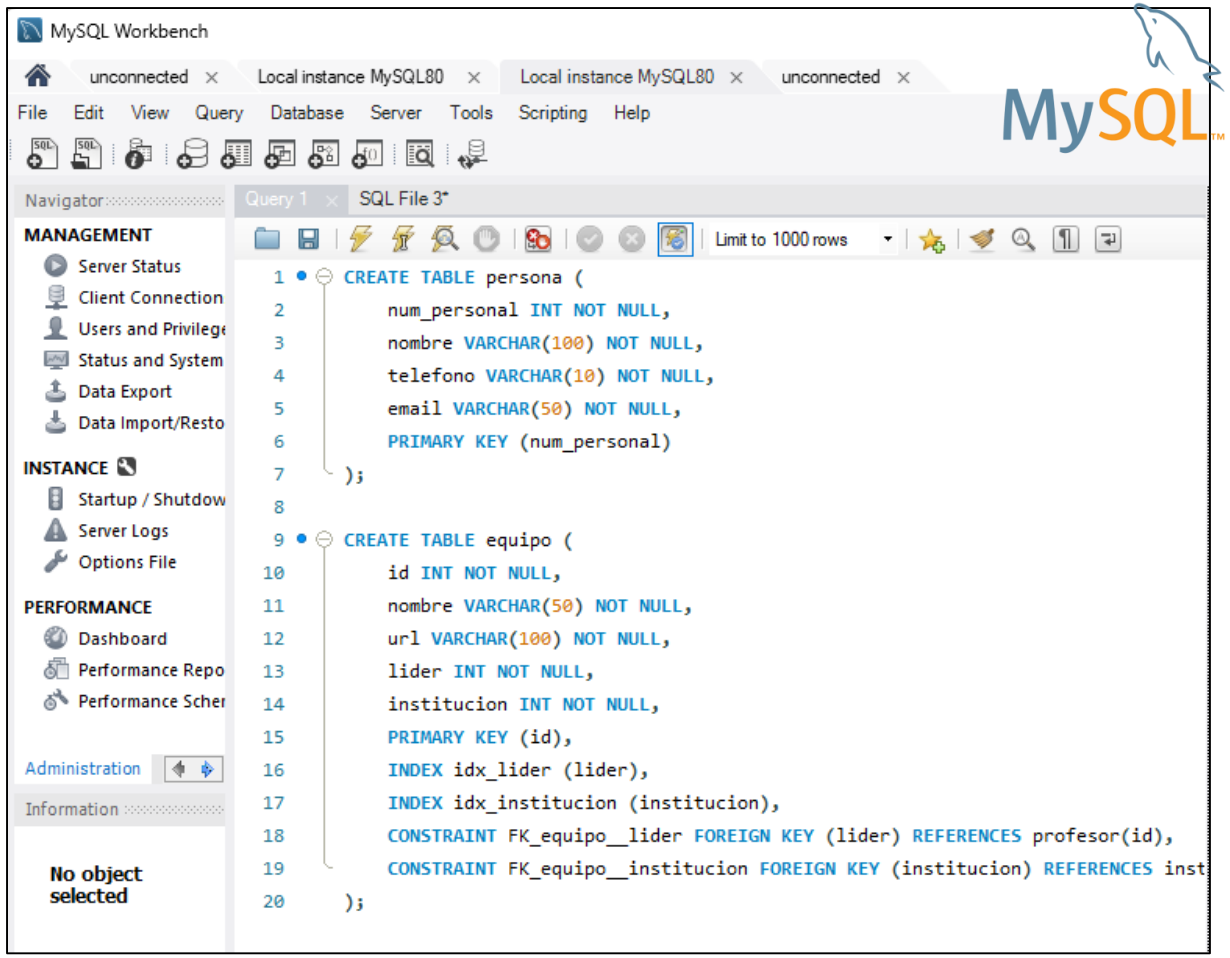



```
1    -- Database: Untitled
2    -- Author:
3 •  CREATE DATABASE untitled
4 ✖      WITH OWNER = postgres
5            ENCODING = 'UTF8'
6            TABLESPACE = proyecto_investigacion
7            LC_COLLATE = 'undefined'
8            LC_CTYPE = 'undefined'
9            CONNECTION LIMIT = -1;
```

Modelado Relacional

3. Modelado Lógico

- Modelo Relacional (ER) → Instrucciones en ***Structured query language (SQL)***

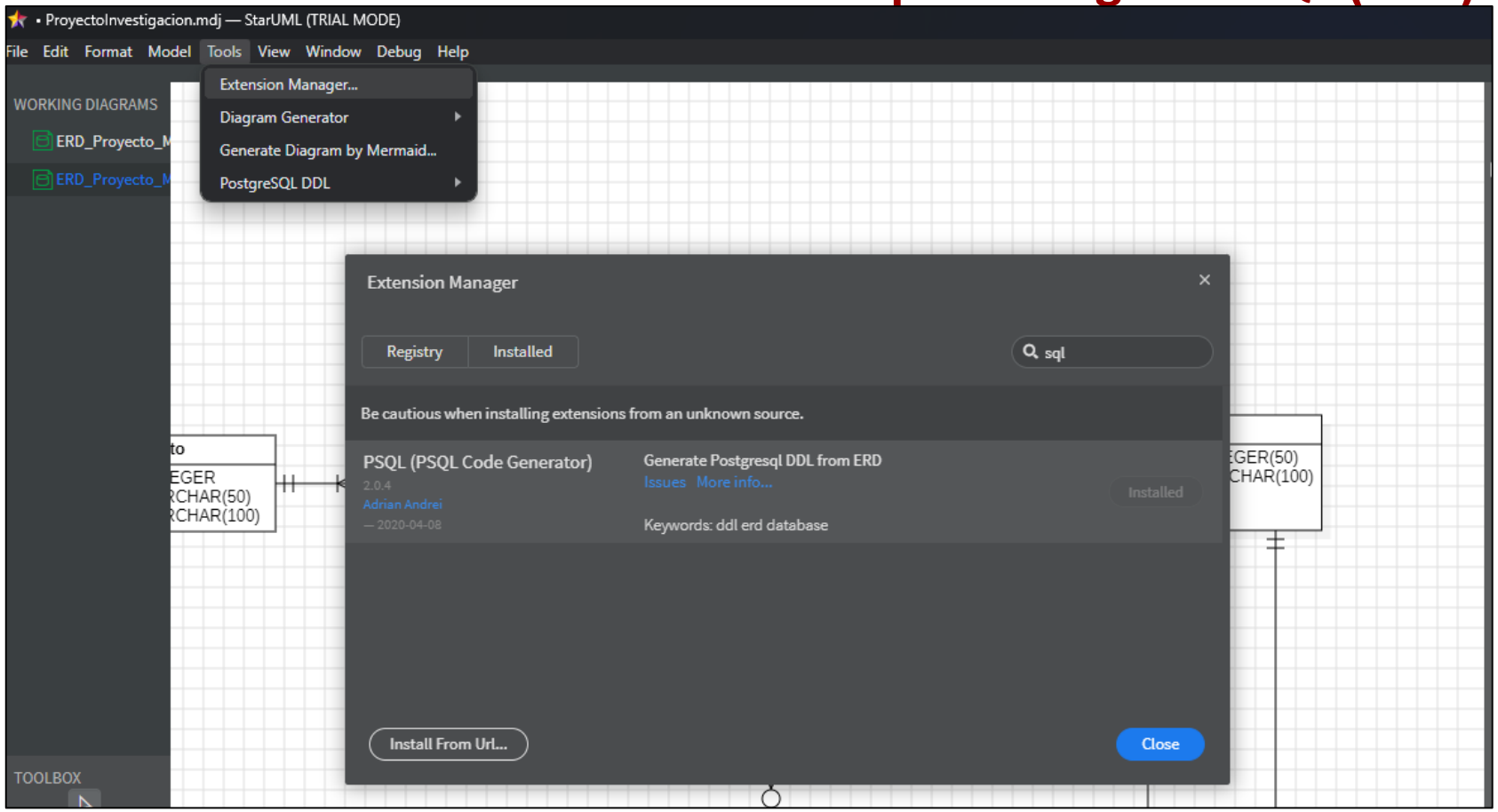



```
CREATE TABLE public.persona (
    num_personal integer NOT NULL,
    nombre varchar(100) NOT NULL,
    telefono varchar(10) NOT NULL,
    email varchar(50) NOT NULL,
    PRIMARY KEY (num_personal)
);


CREATE TABLE public.equipo (
    id integer NOT NULL,
    nombre varchar(50) NOT NULL,
    url varchar(100) NOT NULL,
    lider varchar(50) NOT NULL,
    institucion varchar(50) NOT NULL,
    lider integer NOT NULL,
    institucion integer NOT NULL,
    PRIMARY KEY (id)
);

ALTER TABLE public.equipo ADD CONSTRAINT FK_equipo__lider FOREIGN KEY (lider) REF
ALTER TABLE public.equipo ADD CONSTRAINT FK_equipo__institucion FOREIGN KEY (inst
```

Modelado Relacional

3. Modelado Lógico

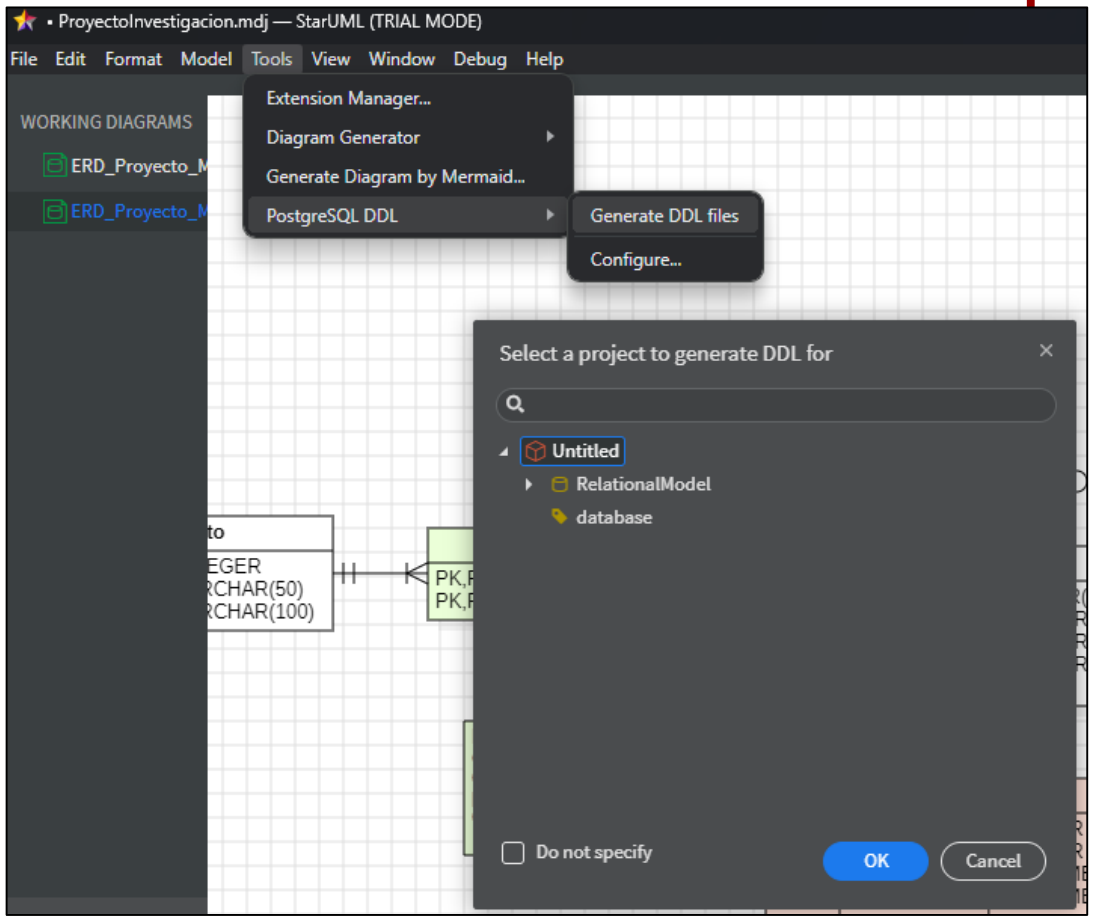
- Modelo Relacional (ER) → Instrucciones en ***Structured query language (SQL)***
  - **Traducción automática de esquema lógico a SQL (DDL)**

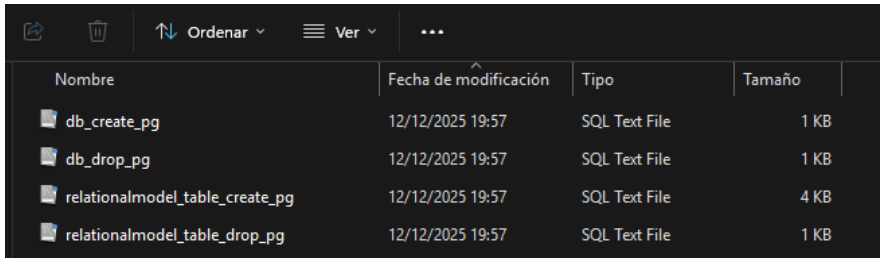


Modelado Relacional

3. Modelado Lógico

- Modelo Relacional (ER) → Instrucciones en ***Structured query language (SQL)***
  - **Traducción automática de esquema lógico a SQL (DDL)**



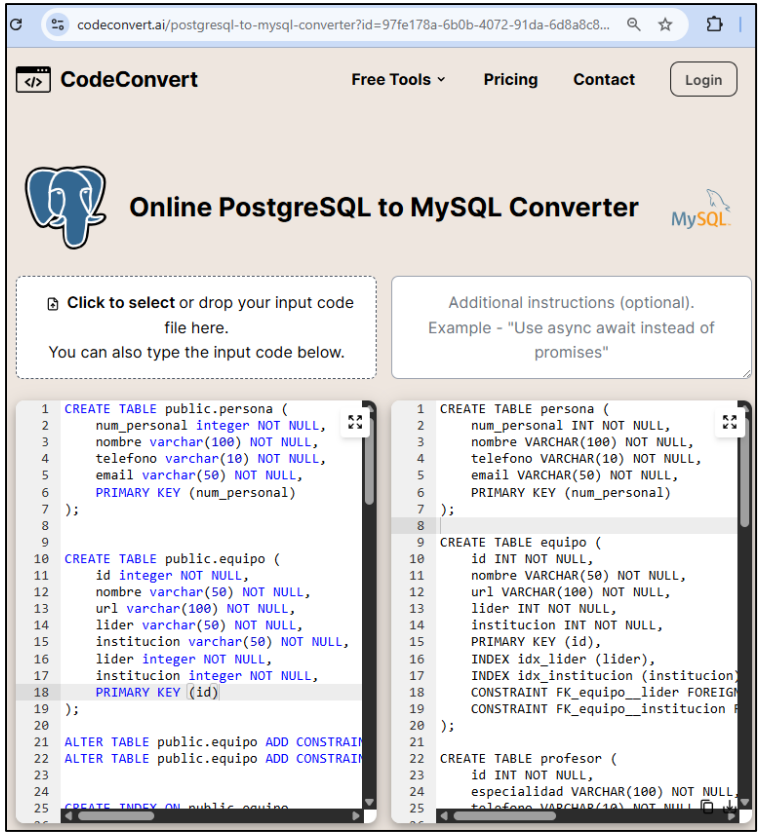



```
CREATE TABLE public.persona (
    num_personal integer NOT NULL,
    nombre varchar(100) NOT NULL,
    telefono varchar(10) NOT NULL,
    email varchar(50) NOT NULL,
    PRIMARY KEY (num_personal)
);


CREATE TABLE public.equipo (
    id integer NOT NULL,
    nombre varchar(50) NOT NULL,
    url varchar(100) NOT NULL,
    lider varchar(50) NOT NULL,
    institucion varchar(50) NOT NULL,
    lider integer NOT NULL,
    institucion integer NOT NULL,
    PRIMARY KEY (id)
);

ALTER TABLE public.equipo ADD CONSTRAINT FK_equipo__lider FOREIGN KEY (lider) REF
ALTER TABLE public.equipo ADD CONSTRAINT FK_equipo__institucion FOREIGN KEY (inst
```

Modelado Relacional

3. Modelado Lógico

- Modelo Relacional (ER) → Instrucciones en ***Structured query language (SQL)***
  - **Traducción automática de esquema lógico a SQL (DDL)**

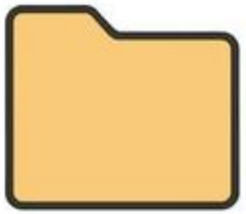


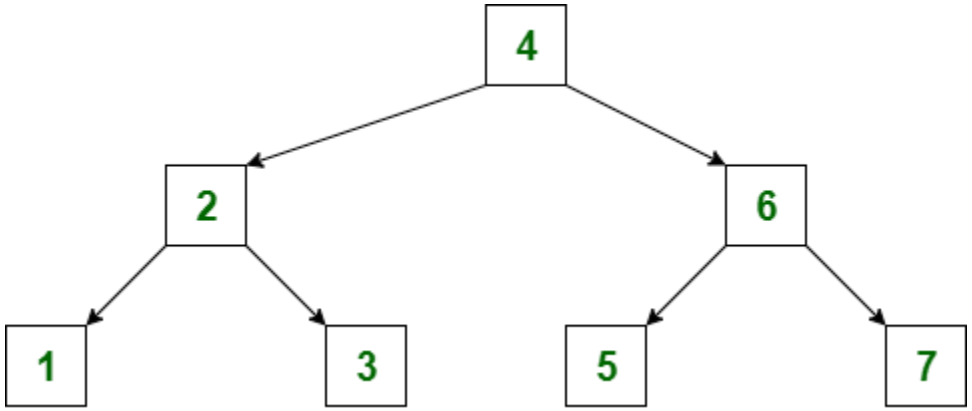
https://www.codeconvert.ai/postgresql-to-mysql-converter

Modelado Relacional

4. Modelado Físico

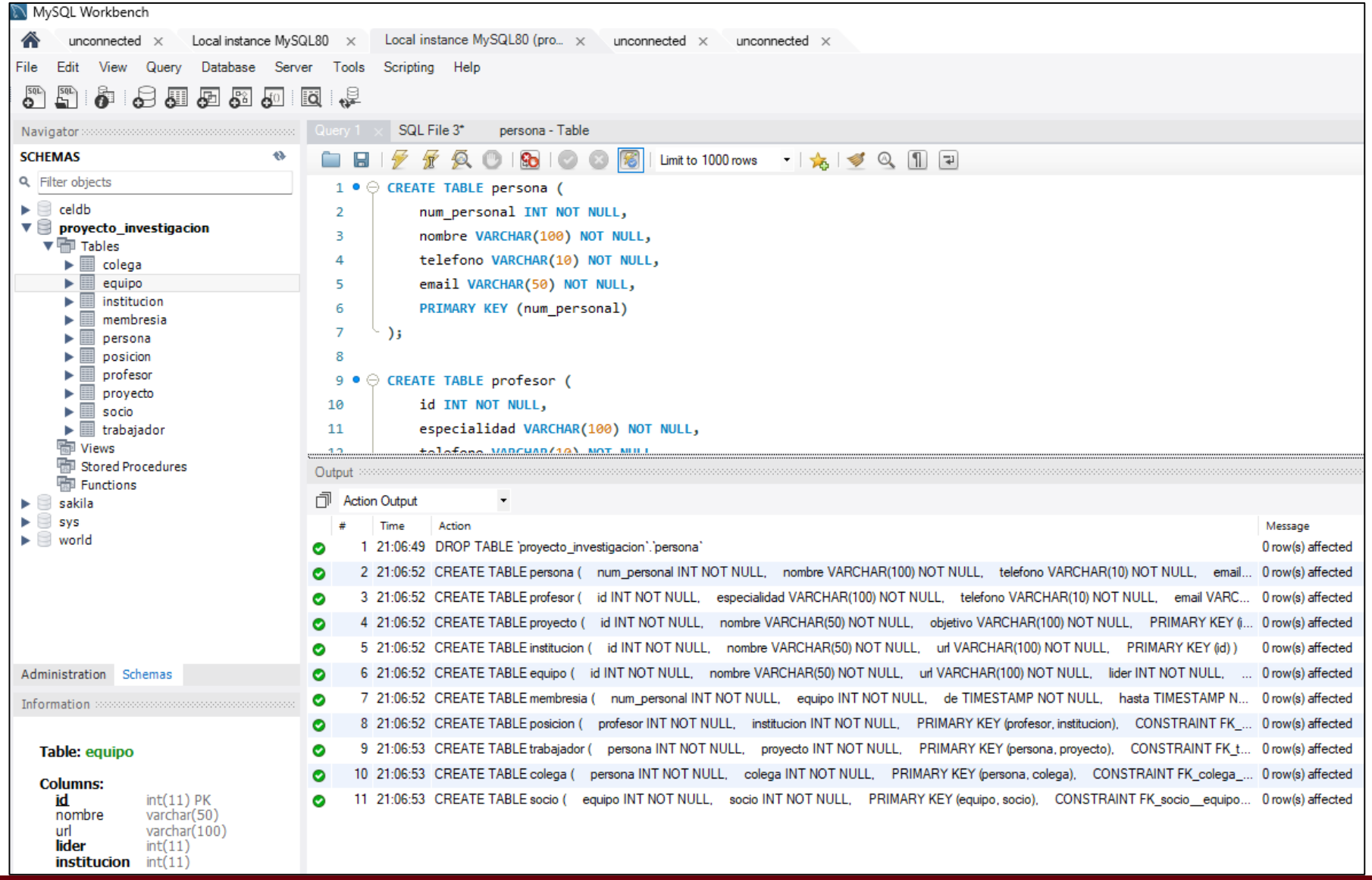
- Especifica **cómo se implementan** las **estructuras lógicas** de **bases de datos** en un **entorno técnico específico**.
- Modelo **Entidad-Relación** (ER) → **Structured query language (SQL)** → **Archivos de datos**, **estructuras de índice** (p. ej., **árboles B+**), etc.
- Ejecutar los **scripts SQL** generados en un cliente del **SGBD elegido**.





Modelado Relacional

4. Modelado Físico



SQL

- ***Structured query language (SQL)***
- **SQL es un lenguaje estándar para acceder y manipular bases de datos.**
  - Significa **Lenguaje de Consulta Estructurado**.
  - Permite acceder y manipular bases de datos.
  - Estándar del Instituto Nacional Estadounidense de Estándares (ANSI) en 1986 y de la Organización Internacional de Normalización (ISO) en 1987.
- **Lenguaje estándar para acceder a bases de datos relacionales**
  - **Definición de datos (DDL)**
    - Creación de esquemas de tablas y restricciones de integridad
  - **Manipulación de datos (DML)**
    - Consultas
    - Inserción, eliminación y actualización de datos
  - Gestión de **transacciones**

SQL

- **¿Qué puede hacer SQL?**
  - ejecutar consultas en una base de datos.
  - recuperar datos de una base de datos.
  - insertar registros en una base de datos.
  - actualizar registros en una base de datos.
  - eliminar registros de una base de datos.
  - crear nuevas bases de datos.
  - crear nuevas tablas en una base de datos.
  - crear procedimientos almacenados en una base de datos.
  - crear vistas en una base de datos.
  - establecer permisos en tablas, procedimientos y vistas.
  - ejecutar transacciones

Data Query Language (DQL)

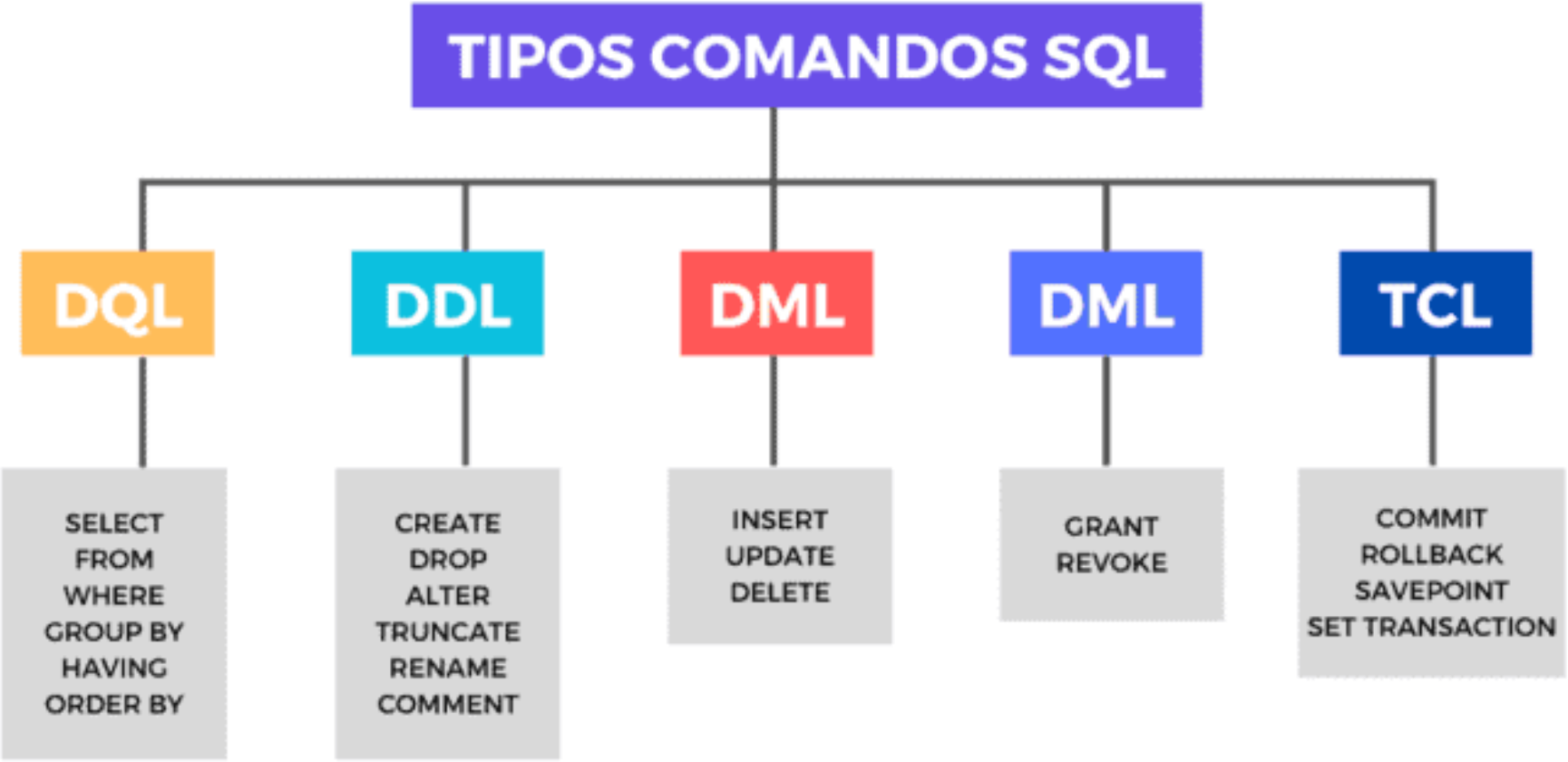
Data Manipulation Language (DDL)

Data Definition Language (DDL)

Data Control Language (DCL)

Transaction Control Language (TCL)

SQL



CodigoNautas

https://codigonautas.com/guia-completa-consultas-sql/

SQL

- **Data Definition Language (DDL)**

```
CREATE DATABASE mi_base_de_datos;
```

```
CREATE SCHEMA mi_esquema;
```

```
CREATE TABLE empleados (
 id_empleado INT PRIMARY KEY,
 nombre VARCHAR(50),
 apellido VARCHAR(50),
 fecha_contratacion DATE
);
```

```
RENAME TABLE tabla_antigua
 TO tabla_nueva;
```

```
ALTER TABLE mi_tabla
 ADD COLUMN edad INT;
```

```
TRUNCATE TABLE mi_tabla;
```

```
DROP TABLE mi_tabla;
```

SQL

- **Data Definition Language (DDL)**
  - **Restricciones de Integridad**
    - **id INT AUTO_INCREMENT PRIMARY KEY:** Crea una **columna id** de tipo **INT** que se **incrementará automáticamente** para cada nuevo registro (**será único**).
    - **FOREIGN KEY (id_producto) REFERENCES productos(id): E**stablece una **clave externa** en la **columna id_producto** que **referencia** a la **columna id** de la tabla **productos.**
    - **CHECK (cantidad > 0):** Garantiza que el **valor de la columna cantidad** sea siempre **mayor que cero.**

```sql
CREATE TABLE pedidos (
 id INT AUTO_INCREMENT PRIMARY KEY,
 id_producto INT,
 cantidad INT,
 FOREIGN KEY (id_producto) REFERENCES productos(id),
 CHECK (cantidad > 0)
);
```

SQL

- **Data Manipulation Language (DML)**

```
INSERT INTO nombre_de_la_tabla (columna1, columna2, ...)
VALUES (valor1, valor2, ...);
```

```
UPDATE nombre_de_la_tabla
 SET columna1 = valor1, columna2 = valor2, ...
 WHERE condicion;
```

```
DELETE FROM nombre_de_la_tabla
 WHERE condicion;
```

SQL

- **Data Control Language (DCL)**

Otorga a un usuario o rol privilegios específicos sobre un objeto de base de datos.

```
GRANT SELECT, INSERT ON empleados TO usuario1;
```

Revoca privilegios específicos de un usuario o rol sobre un objeto de base de datos.

```
REVOKE SELECT, INSERT ON empleados FROM usuario1;
```

SQL

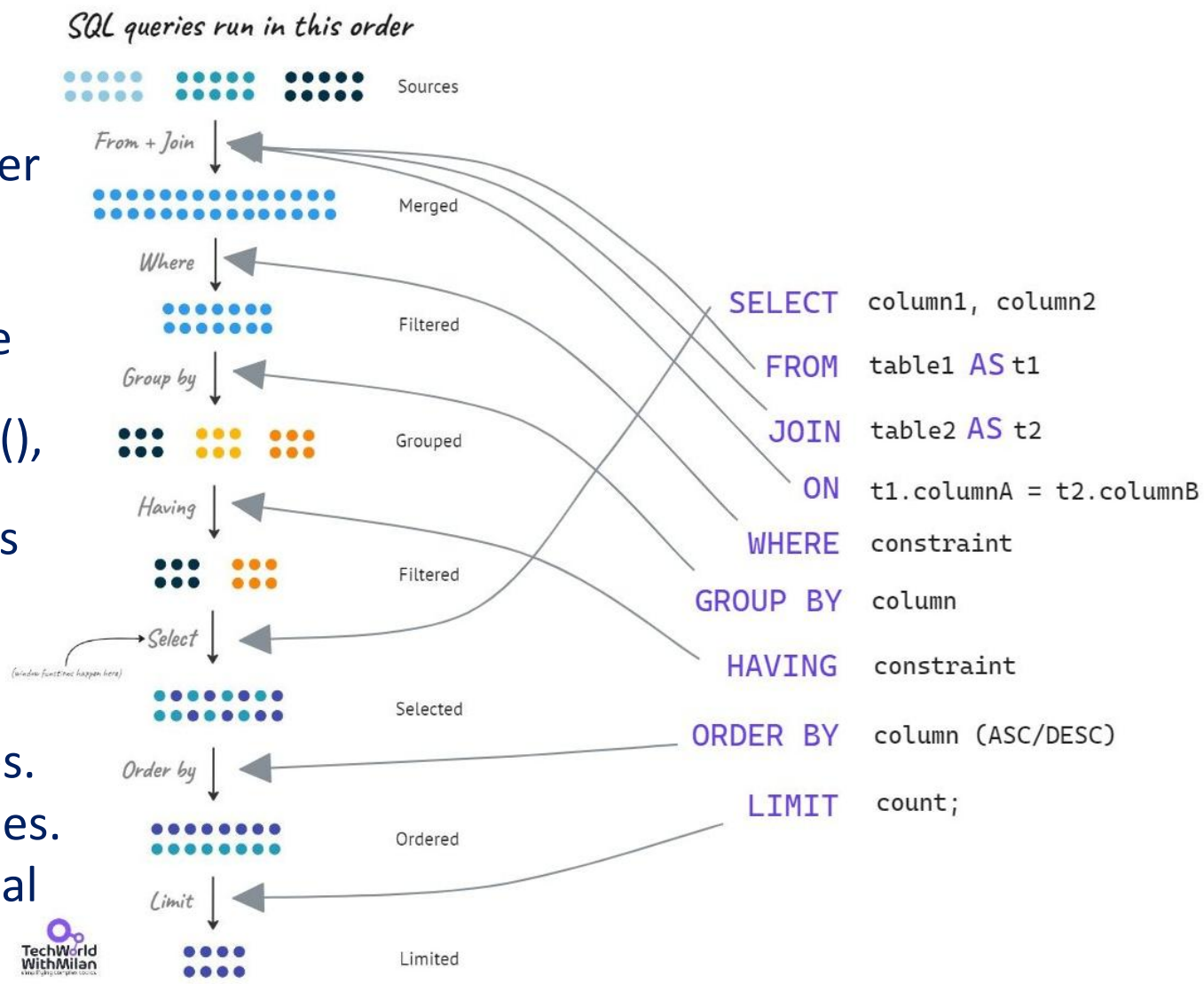
- **Data Query Language (DQL)**

```
SELECT DISTINCT column1, column2, aggregate_function(column3), ...
FROM table_name [INNER | LEFT | RIGHT | FULL] JOIN table_name2
                                ON (table_name1.column = table_name2.column)
WHERE condition1 [AND | OR condition2]
GROUP BY column1 [, column2]
HAVING aggregate_function(column3) operator value
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC], ...
[LIMIT number_of_rows | OFFSET number_of_rows];
```

SQL

- **Data Query Language (DQL)**
  - **Orden ejecución de consulta SQL:**

1. **FROM:** se unen las tablas para obtener los datos base.
2. **WHERE:** se filtran los datos base.
3. **GROUP BY:** se agrupan los datos base filtrados. A menudo se utiliza con funciones agregadas (**COUNT**(), **MAX**(), **MIN**(), **SUM**(), **AVG**()) para agrupar el conjunto de resultados por una o más columnas.
4. **HAVING:** se filtran los datos base agrupados.
5. **SELECT:** se devuelven los datos finales.
6. **ORDER BY:** se ordenan los datos finales.
7. **LIMIT:** los datos devueltos se limitan al número de filas.



https://newsletter.techworld-with-milan.com/p/how-to-learn-sql

SQL

- **Data Query Language (DQL)**
  - **Operadores de Comparación**

Operador	Descripción
=	Comprueba la igualdad entre dos valores.
<, >	Comprueban si un valor es menor o mayor que otro.
<=, >=	Comprueban si un valor es menor o igual, o mayor o igual que otro.
!= o <>	Comprueban si dos valores son diferentes.
LIKE	Comprueba si una cadena coincide con un patrón utilizando caracteres especiales % para cualquier número de caracteres y _ para un solo carácter

SQL

- **Data Query Language (DQL)**
  - **Operadores de Aritméticos**

Operador	Descripción
+	Suma dos valores.
–	Resta un valor de otro.
*	Multiplica dos valores.
/	Divide un valor entre otro.
%	Módulo. Devuelve el resto de la división de un valor entre otro.

SQL

- **Data Query Language (DQL)**
  - **Operadores Lógicos**

Operador	Descripción
AND	Devuelve TRUE si ambas condiciones son verdaderas, de lo contrario, devuelve FALSE.
OR	devuelve TRUE si al menos una de las condiciones es verdadera, de lo contrario devuelve FALSE.
NOT	invierte el valor de una condición (convierte una condición verdadera en falsa y viceversa)

SQL

- **Data Query Language (DQL)**

```
USE nombre_de_la_base_de_datos;
```

```
SELECT columna1, columna2, ...
FROM nombre_de_la_tabla;
```

```
SELECT *
FROM empleados;
```

Recupera **todas las filas y columnas** de la tabla especificada

```
SELECT columna1, columna2, ...
FROM nombre_de_la_tabla
WHERE condicion;
```

condición/predicado que deben cumplir los **datos** (verdadero o falso)

```
SELECT columna1, columna2, ...
FROM nombre_de_la_tabla
GROUP BY columna1, columna2, ...;
```

agrupar filas por los valores de **columnas** específicas

SQL

- **Data Query Language (DQL)**

```sql
SELECT nombre, apellido, salario
FROM empleados
WHERE salario > 50000;
```

selecciona empleados con un **salario superior** a 50.000

```sql
SELECT nombre, apellido, salario
FROM empleados
WHERE salario BETWEEN 50000 AND 80000;
```

seleccionar **valores dentro de un rango específico**

```sql
SELECT nombre, apellido
FROM clientes
WHERE nombre LIKE 'A%';
```

Seleccionar **todos los clientes** cuyo **nombre empiece por la letra "A"**

```sql
SELECT nombre_del_producto, categoria
FROM productos
WHERE categoria IN ('Electronica', 'Ropa');
```

compara un valor con un conjunto de valores enumerados en una lista

```sql
SELECT nombre, apellido
FROM clientes
WHERE numero_de_telefono IS NULL;
```

selecciona **filas en las que falta un valor de columna** (es NULL)

SQL

- **Data Query Language (DQL)**
  - **Funciones de Agregación**
    - **Calcula resúmenes para cada combinación única de valores**

Función	Descripción
COUNT (expr)	Cuenta el número de filas o valores no nulos
SUM(expr)	Suma total de valores numéricos en una columna
AVG(expr)	Calcula el promedio de una columna numérica
MIN(expr)	Obtiene el valor mínimo
MAX(expr)	Obtiene el valor máximo

SQL

- **Data Query Language (DQL)**

```
SELECT COUNT(*)
FROM empleados;
```

cuenta el número de empleados

```
SELECT COUNT(nombre)
FROM empleados;
```

cuenta el número de empleados con nombre no nulo

```
SELECT SUM(salario)
FROM empleados;
```

calcula la suma de los valores de la columna especificada

```
SELECT AVG(edad)
FROM empleados;
```

calcula la media de los valores de la columna especificada

```
SELECT MAX(precio)
FROM productos;
```

devuelve el valor máximo de la columna especificada

```
SELECT MIN(stock)
FROM inventario;
```

devuelve el valor mínimo de la columna especificada

SQL

- **Data Query Language (DQL)**
  - **Puntos Clave**:

	Descripción
ORDER BY	Ordena el conjunto de resultados de una consulta SELECT en un orden específico (ASC o DESC), basándose en una o más columnas.
GROUP BY	Esencial para desglosar agregaciones por categorías Permite aplicar **funciones de agregación** a **cada grupo por separado**
HAVING	Filtra los resultados de las funciones agregadas, a diferencia de WHERE que filtra filas antes de agrupar.
DISTINCT	Puede usarse dentro de la función (ej. COUNT(DISTINCT columna)) para contar o sumar solo valores únicos.

SQL

- **Data Query Language (DQL)**

```
SELECT columna1, columna2, ...
FROM nombre_de_la_tabla
ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC], ...;
```

```
SELECT id_pedido, fecha_pedido, id_cliente
FROM pedidos
ORDER BY fecha_pedido ASC;
```

ordena los pedidos por fecha en orden ascendente.

```
SELECT id_pedido, fecha_pedido, id_cliente
FROM pedidos
ORDER BY fecha_pedido;
```

```
SELECT id_pedido, fecha_pedido, id_cliente
FROM pedidos
ORDER BY fecha_pedido DESC;
```

ordena los pedidos por fecha en orden descendente.

SQL

- **Data Query Language (DQL)**

```
SELECT columna1, columna2, ..., funcion_de_agregacion(columna)
FROM nombre_de_la_tabla
GROUP BY columna1, columna2, ...;
```

```
SELECT categoria, COUNT(*) AS
numero_de_productos
FROM productos
GROUP BY categoria;
```

Asigna un **alias** más legible a una columna o tabla

Cuenta el **total** de productos **por cada categoría** de producto

```
SELECT MONTH(fecha_de_venta) AS mes,
   SUM(importe_de_venta) AS ventas_totales
FROM ventas
GROUP BY MONTH(fecha_de_venta);
```

Calcula el **total de ventas por cada mes**

SQL

- **Data Query Language (DQL)**

```
SELECT columna1, columna2, ...,                funcion_de_agregacion(columna)
FROM nombre_de_la_tabla
GROUP BY columna1, columna2, ...
HAVING funcion_de_agregacion(columna) operador value;
```

```
SELECT categoria, AVG(cantidad) AS cantidad_media
FROM productos
GROUP BY categoria
HAVING AVG(cantidad) > 50;
```

filtra las **categorías de productos** cuya **cantidad media** sea **superior a 50**

```
SELECT id_cliente, AVG(importe_pedido)
                        AS importe_medio_pedido
FROM pedidos
GROUP BY id_cliente
HAVING AVG(importe_pedido) > 1000;
```

filtra los **clientes** cuyo **importe medio de pedido** sea **superior a 1000.**

SQL

- **Data Query Language (DQL)**
  - **Subconsultas: Permiten ejecutar una consulta dentro de otra.**
    - **La subconsulta se ejecuta primero y su resultado es utilizado por la consulta principal.**

Tipo	Descripción
VALOR SIMPLE	La subconsulta **retorna exactamente una fila y una columna**. Puede ser usado con **operadores de comparación (=, >, <, …)**
VALORES MULTIPLES	La subconsulta puede **retornar múltiples filas o columnas**. Puede ser usado con **operadores IN, EXISTS, ALL, ANY**

SQL

- **Data Query Language (DQL)**

```
SELECT nombre
FROM empleados
WHERE id_departamento = (
    SELECT id
    FROM departamentos
    WHERE nombre = 'Ventas’
);
```

- La **consulta externa** selecciona los nombres de los empleados de la tabla empleados.
- La **subconsulta** encuentra el **id del departamento** donde **nombre** es igual a ‘**Ventas’**.
- Si se encuentra un **id del departamento**, la **consulta externa incluye** el **nombre** del **empleado** en el **resultado final**.

SQL

- **Data Query Language (DQL)**

```sql
SELECT nombre
FROM clientes
WHERE EXISTS (
 SELECT *
 FROM pedidos
 WHERE pedidos.id_cliente = clientes.id
);
```

- La **consulta externa** selecciona los **nombres** de los **clientes** de la tabla **clientes**.
- La **subconsulta** comprueba si **existe** al menos **un pedido para cada cliente en la tabla pedidos** utilizando la **condición pedidos.id_cliente = clientes.id**.
- Si se encuentra **al menos un pedido para el cliente actual**, la subconsulta devuelve una fila, el operador EXISTS devuelve TRUE. Entonces, la **consulta externa** incluye el **nombre del cliente** en el **resultado final**.

SQL

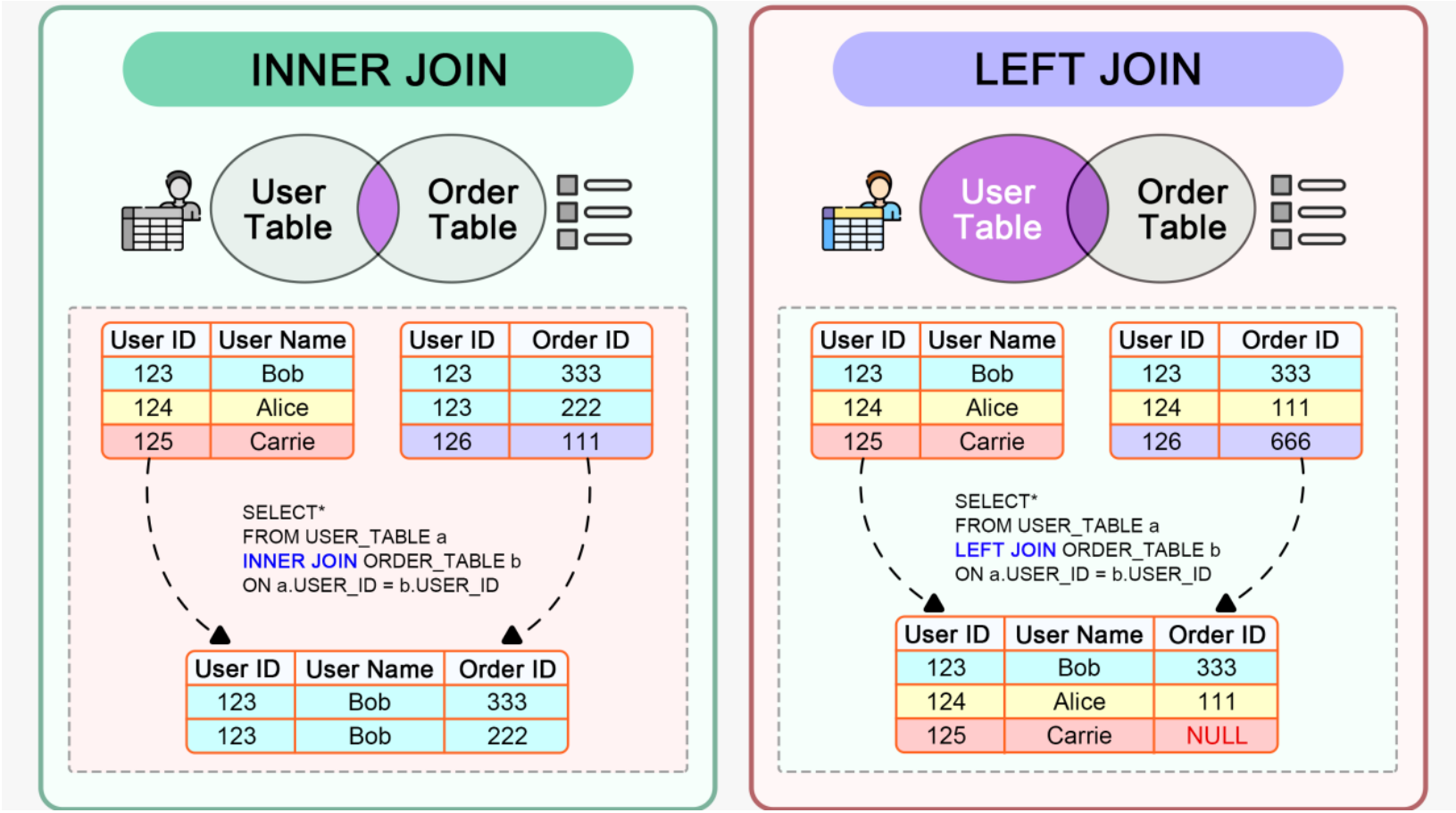
- **Data Query Language (DQL)**
  - **SQL Join**: Combina datos de **dos o más tablas** basándose en una **columna relacionada**.

```
SELECT column_list
FROM table1 [JOIN TYPE] JOIN table2
ON table1.column_name = table2.column_name;
```

Tipo	Descripción
INNER JOIN	Recupera **filas** con **valores coincidentes** en **ambas tablas**.
LEFT JOIN	Recupera **todas las filas de la tabla izquierda** y las **filas coincidentes de la tabla derecha**.
RIGHT JOIN	Recupera **todas las filas de la tabla derecha** y las **filas coincidentes de la tabla izquierda**.
FULL OUTER JOIN	Combina los resultados de LEFT JOIN y RIGHT JOIN.

SQL

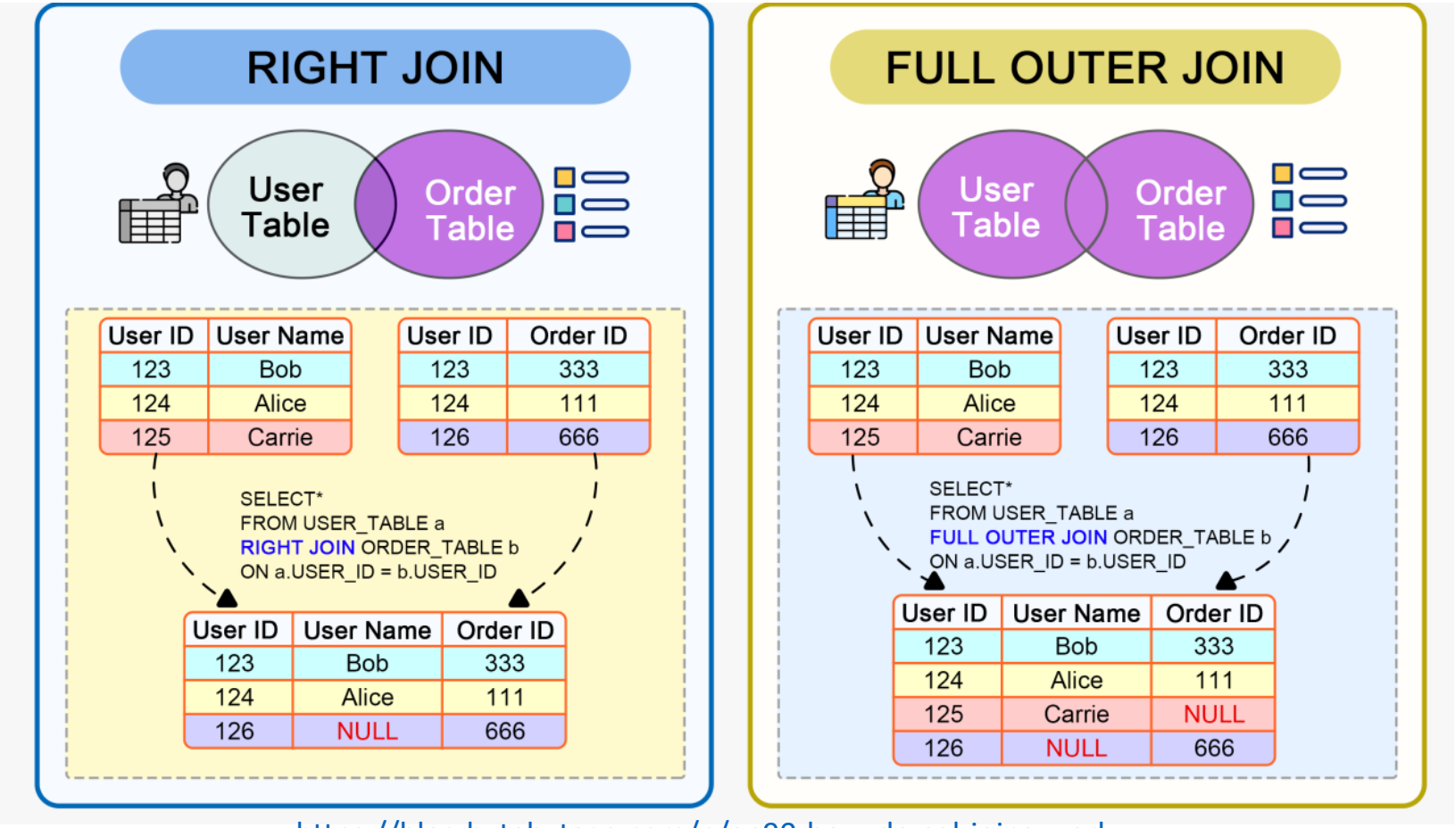
- **Data Query Language (DQL)**
  - **SQL Join**: Combina datos de **dos o más tablas** basándose en una **columna relacionada**.



https://blog.bytebytego.com/p/ep90-how-do-sql-joins-work

SQL

- **Data Query Language (DQL)**
  - **SQL Join**: Combina datos de **dos o más tablas** basándose en una **columna relacionada**.



https://blog.bytebytego.com/p/ep90-how-do-sql-joins-work

Conclusión

- Modelo **relacional**
  - Datos almacenados en tablas planas
  - Adecuado para operaciones por lotes con uso intensivo de datos
- Modelo de **objetos**
  - Datos almacenados como grafos de objetos
  - Adecuado para el acceso de navegación individual a las entidades
- La **elección** de un tipo de Modelo, SGBD, Lenguaje de Consulta depende de la **situación (dominio), características de los datos y tipo de aplicación** a ser desarrollada

REFERENCIAS

Bibliografía

- Laudon, K. C., & Laudon, J. P. (2022). **Management information systems: Managing the digital firm.** Pearson Educación**.**
- Laudon, K. C., & Laudon, J. P. (2023). **Essentials of management information systems.** Pearson.
- Lake P., and Crowther, P. (2013). **Concise Guide to Databases: A Practical Introduction (1th edition).**
- Nield, T. (2016). **Getting Started with SQL.** " O'Reilly Media, Inc.".
- DeBarros, A. (2022). **Practical SQL: A Beginner's Guide to Storytelling with Data.** no starch Press.
- Kleppmann, M. (2017). **Designing Data Intensive Applications.** O'Reilly.

REFERENCIAS

Recursos

- https://www.w3schools.com/sql/sql_intro.asp
- https://codigonautas.com/guia-completa-consultas-sql/
- Martin Svoboda. Czech Technical University in Prague, Faculty of Electrical Engineering. **B0B36DBS: Database Systems**. https://www.ksi.mff.cuni.cz/~svoboda/courses/172-B0B36DBS/
- Elaine Faria. UFU – Brasil. **Banco de Dados 1.** https://www.facom.ufu.br/~elaine/disc/BD/BD.html
- IBM. **Structured Query Language**. https://cdncontribute.geeksforgeeks.org/wp-content/uploads/SQL-Manual.pdf
- IBM. **Guide to SQL: Tutorial**. https://www.ibm.com/docs/en/informix-servers/14.10.0?topic=programming-guide-sql-tutorial